

Architettura degli elaboratori

Simone Paolo Petta

Secondo semestre a.a. 2020/21

Contents

1	Introduzione	7
1.1	Software	7
1.2	Hardware	7
1.3	MIPS	7
2	Intuizione	9
3	Rappresentazione dell'informazione	9
4	Rappresentazione binaria	9
4.1	Quali informazioni consideriamo	9
5	Rappresentazione dei naturali	10
5.1	Sistemi di numerazione	10
5.1.1	Sistema di numerazione posizionale	10
5.2	Sistema di numerazione in base B	10
5.3	Conversione di base	11
5.3.1	Conversione da base 10 a B	11
5.4	Rappresentazione esadecimale	11
5.5	Conversione da base 2 a base 16	11
5.6	Operazione di somma	12
5.7	Rappresentazione di un numero in base B con K cifre	12
5.8	Operazione di somma con K cifre	12
5.9	Misurare la quantita' di dati	12
6	Rappresentazione dei numeri interi con segno	13
7	Codifica	13
7.1	Rappresentazione in modulo e segno	13
7.1.1	Intervallo di rappresentazione	13
7.1.2	Svantaggi	13
7.2	Rappresentazione in complemento a 2	13
7.2.1	Intervallo di rappresentazione	14

7.3	Rappresentazione in Ca2 di un numero	14
7.4	Regole pratiche per facilitare i calcoli	14
7.5	Addizione/sottrazione in Ca2	15
7.6	Overflow nella addizione in Ca2	15
8	Conversione da base 10 a base 2	16
8.1	Algoritmo	16
8.2	Numero di cifre illimitato	16
8.3	Approssimazione di un numero	16
8.4	Conversione da base 2 a base 10	17
8.5	Rappresentazione con N cifre	17
9	Virgola fissa	17
9.1	Esempio	17
10	Virgola mobile (floating point)	18
10.1	Forma normalizzata	18
10.2	Standard IEEE-754	19
10.2.1	Configurazioni particolari	19
11	Rappresentazione dei caratteri	20
11.1	Standard ASCII	20
11.1.1	Repertorio	20
11.1.2	Codice	20
12	La famiglia ISO 8859	21
12.1	Unicode	21
12.2	Codifica	21
13	Introduzione	22
13.1	Brevi note storiche	22
14	Circuito digitale	22
14.1	Circuiti combinatori	22
14.2	Tabella di verita' di una funzione booleana	23
15	Algebra di Boole	23
15.1	Operatori logici	24
15.2	Espressioni booleane	24
15.2.1	Regole di precedenza degli operatori	24
15.3	Dall'espressione alla tabella di verita' di una funzione	25
15.4	Proprieta' degli operatori	25
15.5	Espressioni equivalenti	26
15.5.1	Esempio	26
15.6	Operatori funzionalmente completi	27
15.7	Forme Canoniche	27
15.7.1	Prima forma canonica	27

15.7.2 Esempio	27
15.8 Porte logiche e simboli funzionali	28
15.9 Porte universali NAND e NOR	28
15.10 Da funzione in forma algebrica al circuito	29
15.11 Convenzioni grafiche	29
15.12 Analisi di un circuito combinatorio	29
15.12.1 Esempio1	30
15.12.2 Esempio2	30
15.13 Sintesi di un circuito combinatorio	30
15.13.1 Esempio : passo 1	31
15.13.2 Esempio: passo 2	31
15.13.3 Esempio: passo 3	31
15.13.4 Esempio: passo 4	32
15.14 Semplificazione del circuito	32
15.15 Propagazione del segnale: aspetti fisici	32
16 Circuiti combinatori con uscita multipla	33
16.1 Esempio	33
17 Blocchi funzionali combinatori	33
17.1 XOR	34
17.2 XNOR	34
17.3 Espressioni algebriche	35
17.4 Comparatore di uguaglianza	35
17.5 Decoder N-M	35
17.5.1 Decoder 2-4	36
17.6 Multiplexer MUX	36
17.6.1 Multiplexer a 2 ingressi	37
17.6.2 Multiplexer a piu' vie	37
17.7 Circuito sommatore	37
17.7.1 Half Adder	38
17.7.2 Full Adder	39
17.8 Sommatore di parole di N bit	40
18 Introduzione	41
19 Circuiti sequenziali	41
19.1 Evoluzione temporale del segnale	41
19.2 Retroazione	42
20 Elemento di memoria	42
21 Circuito Latch di tipo SR	43
21.1 Stato indefinito	43
21.2 Segnali anomali	43

22 Circuiti latch asincroni	44
22.1 Latch SR realizzato con porta NOR	44
22.2 Latch SR con porte NAND	44
23 Circuiti sincroni	45
23.1 Clock	45
23.2 Elemento di memoria sincrono: Latch D	45
23.3 Problema della trasparenza	46
24 Flip-Flop	47
24.1 Flip-flop tipo D	47
25 Registri	47
25.1 Un semplice registro	48
26 Introduzione	49
27 Linguaggio macchina	49
28 Modello generale di computer	49
28.1 Processore (CPU)	50
28.2 La memoria centrale	50
28.3 Aspetti tecnologici: memorie	51
28.4 L'architettura dell'insieme delle istruzioni (ISA)	52
28.5 Classi di architetture	52
29 Architettura MIPS	52
29.1 Memoria centrale	52
29.1.1 Disposizione dei byte in una parola	53
29.2 Processore	53
29.2.1 Program Counter (PC)	54
29.3 Co-processori	54
30 Istruzioni linguaggio Assembly	55
30.1 Esempio	55
30.2 Traduzione dei programmi	55
31 Introduzione	56
32 Regole sintattiche di base	56
33 Classi di istruzioni MIPS	56
33.1 Istruzioni aritmetiche	56
33.1.1 Esempio1	57
33.1.2 Esempio2	57
33.2 Tipi di dato	57
33.3 Overflow	58

34	Corrispondenza con linguaggio ad alto livello	58
35	Istruzioni di scorrimento e logiche	58
35.1	Istruzioni di scorrimento SLL/SRL	58
35.2	Esempio	59
35.3	Istruzioni logiche AND/ANDI	59
35.3.1	Esempio	59
35.4	Istruzioni logiche OR/ORI	59
35.4.1	Esempio	60
36	Istruzioni di trasferimento dati	60
36.1	Memoria MIPS	60
37	Ciclo fetch -execute	60
37.1	Program Counter	61
38	Istruzioni in sequenza	61
39	Istruzioni di salto	61
39.1	Istruzioni di salto condizionato: beq	61
39.1.1	Etichette	62
39.2	Istruzioni di salto condizionato: bne	62
39.2.1	Esempio	62
39.3	Istruzione di confronto SLT	63
39.3.1	Esempio	63
39.4	Istruzioni di salto incondizionato J	63
40	Rappresentazione binaria delle istruzioni	63
41	Formato delle istruzioni MIPS	64
41.1	Istruzioni in formato R	64
41.1.1	Esercizio	66
41.1.2	Istruzioni di scorrimento	66
41.2	Formato I	67
41.2.1	Trasferimento dati	67
41.2.2	Esercizio	67
41.2.3	Istruzioni aritmetiche e logiche con op. immediato	68
41.2.4	Nota sulle costanti	70
41.2.5	Istruzioni di salto condizionato	70
41.3	Formato J	71
41.3.1	Costruzione dell'indirizzo	71
42	Riepilogo	71
43	Introduzione	73
44	CPU: unita' di controllo e unita' di elaborazione	73

45	Unita' di elaborazione	73
46	Temporizzazione	74
47	CPU singolo ciclo	75
48	Realizzazione di un'unita' di elaborazione	75
49	Fetch di un'istruzione	75
49.1	Fase di decodifica	76
49.2	Esecuzione istruzioni formato R	76
49.3	Banco dei registri (register file)	76
49.3.1	Lettura dei registri	77
49.3.2	Scrittura di un registro	77
49.4	ALU: unita' aritmetico logica	78
49.4.1	Esecuzione di una istruzione di tipo R: lettura registri . .	78
49.4.2	Esecuzione di una istruzione di tipo R: ALU	78
49.4.3	Esecuzione di una istruzione di tipo R: scrittura risultato	79
50	Istruzioni di trasferimento dati: lw (sw)	79
50.1	Unita' funzionali coinvolte (lw/sw)	79
51	Istruzione lw	80
51.1	lw: calcolo indirizzo memoria	80
51.2	lw: lettura dato e scrittura	80
51.3	lw: schema complessivo	80
52	Istruzione sw	80
53	Schema per istruzioni tipo R e lw/sw	81
54	Istruzione beq	82
54.1	Istruzione beq: calcolo indirizzo di salto	82
54.2	beq: confronto dei registri	83
54.3	Indirizzo da scrivere nel PC	83
55	Istruzione jump	83
56	Unita' di Controllo (UC)	85
56.1	Unita' di controllo della ALU	85
56.2	Unita' di controllo principale (UC)	86

1 Introduzione

Il computer e' una semplice astrazione:

- Hardware: circuiti digitali e componenti fisici.
- Software di sistema: sistema operativo, ovvero un'insieme di programmi che nascondono la visibilita' dell'hardware, organizzano i dati ma non abbiamo la consapevolezza di come fisicamente i dati vengano organizzati.
- Software applicativo: Insieme di tutti i programmi usati dall'utente finale che utilizzano le funzionalita' messe a disposizione dal sistema operativo.

Nell'organizzazione di un elaboratore esistono diversi livelli di dettaglio con cui puo' essere osservato lo stesso computer.

1.1 Software

- Linguaggio macchina: e' un linguaggio binario (utilizza simboli 0 e 1) specifico per la macchina hardware, sono i soli programmi che possono essere eseguiti dal processore.
- Assembly: linguaggio macchina espresso in forma simbolica, si utilizzano dei simboli per esprimere le istruzioni. Per essere eseguito dev'essere tradotto in linguaggio macchina.
- Linguaggio di programmazione ad alto livello (go, java ...)

1.2 Hardware

- CPU (processore): dove vengono eseguite tutte le istruzioni
- Memoria Centrale: dove viene trasferito il programma tradotto in linguaggio macchina.
- I/O (dispositivi di Ingresso/Uscita)

1.3 MIPS

Storia dei calcolatori:

- Fase pionieristica durante la seconda guerra mondiale. Progetto ENIAC e' il primo calcolatore elettronico;
- Sviluppi commerciali: sistemi usati in ambiti commerciali (banche).
- Microprocessori e Persona Computer: le macchine sono diventate di dimensioni piu' ridotte ed ad un costo accessibile per le persone.

- RISC (vs. CISC): modalita' di progettazione dei processori per cui le istruzioni sono poche e semplici. La semplicita' si traduce in prestazioni piu' elevate.

Il processore MIPS viene usato in sistemi embedded, in console per videogiochi inoltre e' molto usato per scopi didattici. Il processore RISC leader di mercato e' ARM.

2 Intuizione

Ci possono essere molteplici rappresentazioni della stessa informazione. Una rappresentazione condivisa agevola lo scambio di informazioni. L'esigenza di avere una rappresentazione condivisa porta alla creazione di standard. L'informazione e' la semantica e la rappresentazione e' la sintassi.

3 Rappresentazione dell'informazione

Usiamo il termine "informazione" per indicare una conoscenza elementare, ad esempio una quantita', un nome. Un sinonimo di informazione e' "dato". **Rappresentare** l'informazione significa stabilire una corrispondenza fra l'insieme di informazioni e una serie di simboli di un alfabeto $A = a_1, \dots, a_n$. Definire una corrispondenza significa creare una coppia di simbolo e significato ad esso attribuito. Il sistema di simboli usati per rappresentare l'informazione costituisce il codice:

- **codifica**: associare ad una informazione una serie di simboli, tramite il codice e l'insieme di composizione;
- **decodifica**: determinare l'informazione associata ad una serie di simboli

4 Rappresentazione binaria

Si utilizzano due soli simboli (differenti) convenzionalmente indicati come 0 e 1. La cifra binaria e' il **bit** (binary digit). Il bit e' la più piccola unità di informazione elaborabile da un calcolatore e corrisponde allo stato di un dispositivo fisico che viene interpretato come 0 o 1. Per motivi tecnologici distinguere tra due valori di una grandezza fisica e' più semplice che non ad esempio tra dieci valori. Una sequenza di 8 bit forma 1 **byte**. Meno usato e' il termine **nibble**, sequenza di 4 bit. Esempio:

- 1010 0100 un byte costituito da 2 nibble

4.1 Quali informazioni consideriamo

- Numeri naturali $N=\{0,1,2,\dots\}$. In informatica, i numeri naturali sono anche detti interi senza segno (unsigned integer).
- Numeri relativi $Z=\{\dots-2, -1, 0, +1, +2,\dots\}$, anche detto interi con segno (signed integer)
- Numeri con componente frazionaria. Esempio: 4,784. In informatica sono anche detti numeri real o float, non sono i numeri reali, sono numeri con la parte frazionaria finita.
- caratteri

5 Rappresentazione dei naturali

5.1 Sistemi di numerazione

Un sistema di numerazione è costituito da un insieme di cifre e da regole per la composizione dei numeri. Può essere: posizionale, non-posizionale. Sistema non-posizionale: ogni cifra rappresenta sempre lo stessa quantità. Ad esempio il sistema di numerazione romano e' non-posizionale.

5.1.1 Sistema di numerazione posizionale

La quantità rappresentata dalla cifra dipende dalla posizione nella sequenza, quindi lo stesso simbolo puo' avere significato diverso. Il concetto chiave è quello di 'base' del sistema di numerazione. La base è definita dal numero di cifre (e implicitamente dall'insieme di cifre) che costituisce l'alfabeto (≥ 2).

Numeri in base diversa:

- Base 10: 931_{10}
- Base 8: 437_8
- Base 2: 1010_2
- Base 16: $A17B_{16}$

Insieme delle cifre (alfabeto)	Base	Sistema di numerazione
0,1,2,3,4,5,6,7,8,9	10	Decimale
0,1	2	Binario
0,1,2,3,4,5,6,7	8	Ottale
0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F	16	Esadecimale

Table 1: Basi piu' importanti

5.2 Sistema di numerazione in base B

Sia B la base e si consideri l'insieme delle cifre corrispondenti: $\{0,1,\dots,B-1\}$. Un numero N in base B è rappresentato da una sequenza di cifre:

$$N_B = C_{k-1} \dots C_1 C_0 \text{ con } C_i \in \{0, \dots, B-1\} \quad (1)$$

Per esprimere il numero N in base 10 a partire dalla base B si calcola: $N_{10} = \sum_{i=0}^{k-1} c_j \cdot B^i$

$$N_{10} = C_{k-1} B^{k-1} + \dots + C_1 B^1 + C_0 B^0 \quad (2)$$

$$N_{10} = C_{k-1} B^{k-1} + \dots + C_1 B^1 + C_0 \quad (3)$$

Esempio:

$$B = 10, N_{10} = 424 \rightarrow 4 \cdot 10^2 + 2 \cdot 10^1 + 4 \cdot 10^0 \quad (4)$$

$$B = 2, N_2 = 10101 \rightarrow N_{10} = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 16 + 4 + 1 = 21 \quad (5)$$

5.3 Conversione di base

L'operazione di conversione di base converte un numero da una base ad un'altra. Abbiamo già visto la conversione da base B a base 10. Ora consideriamo:

- Conversione da base 10 a base B
- Conversione da base B a base 10

5.3.1 Conversione da base 10 a B

Algoritmo di conversione di un numero x in base 10 in base B:

i = 0

Divido (div. intera) il numero x per B

Resto della divisione:

 cifra i-esima in base B

 i = i+1

Quoziente della divisione $\rightarrow$ x

si prosegue fino a che il quoziente x = 0

L'ultimo resto è la cifra più significativa del numero in base B

5.4 Rappresentazione esadecimale

Base = 16. Molto utilizzata in alternativa alla numerazione binaria. Notazioni comunemente utilizzate: $9F_{16}$, $0x9F$, $9fhex$. In base 16 perché ci permette di rappresentare in modo più conciso le rappresentazioni binarie. 4 cifre binarie (nibble) sono tradotte in 1 cifra esadecimale.

5.5 Conversione da base 2 a base 16

Si raggruppano le cifre in gruppi di 4, a partire da destra. Quindi si converte il numero rappresentato da ogni gruppo (compreso fra 0 e 15) in base 16:

- Conversione di 10011011_2

$$1001_2 \rightarrow 9_{16} \quad (6)$$

$$0110_2 \rightarrow 6_{16} \quad (7)$$

$$6B_{16} \quad (8)$$

Viene aggiunto uno 0 per formare un gruppo di 4 persone.

5.6 Operazione di somma

Delle operazioni aritmetiche, ci limitiamo a considerare di somma fra due numeri X e Y in base B . Esempio:

$$1101_2 + 1000_2 \quad (9)$$

L'operazione si effettua sommando le singole cifre degli addendi partendo da destra verso sinistra e tenendo conto dei riporti. Si genera il riporto quando la somma delle cifre supera la base.

5.7 Rappresentazione di un numero in base B con K cifre

Data una base B , supponiamo di poter utilizzare la massimo K di cifre. Quanti e quali numeri posso rappresentare? Con K cifre in base B , possiamo rappresentare B^K numeri, da 0 a $B^K - 1$. L'insieme dei numeri rappresentabili costituisce l'intervallo di rappresentazione:

$$[0, B^K - 1] \quad (10)$$

Ad esempio con $K=3$, $B=10$, l'intervallo di rappresentazione è $[0, 999_{10}]$. Il numero più grande rappresentabile con K cifre in base B è costituito da cifre tutte uguali a $B-1$. Esempio: con $K=5$, $B=7$ il numero più grande è 66666_7 .

5.8 Operazione di somma con K cifre

Si consideri sempre il caso in cui si possono usare K cifre in base B . Consideriamo l'operazione di somma. La somma fra 2 numeri può non essere rappresentabile con K cifre. Esempio: $K=2$, $B=10 \rightarrow 35+80$ non è rappresentabile. In questo caso, si dice che l'operazione di somma determina overflow (trabocco)

5.9 Misurare la quantita' di dati

Prefisso		Prefisso	
KB (Kilobyte)	10^3 byte	Kib (kibibyte)	2^{10} bytes
MB (Megabyte)	10^6 byte	Mib (Mebibyte)	2^{20} bytes
GB (Gigabyte)	10^9 byte	Gib (Gibibyte)	2^{30} bytes
TB (Terabyte)	10^{12} byte	Tib (Tebibyte)	2^{40} bytes
PB (Petabyte)	10^{15} byte	Pib (Pebibyte)	2^{50} bytes

Table 2: Unità di misura

6 Rappresentazione dei numeri interi con segno

$Z = \dots - 2, -1, 0, +1, +2, \dots$ l'insieme dei numeri relativi. In informatica si indicano anche in il termine interi con segno. Nel sistema di numerazione decimale, il numero viene rappresentato in modulo e segno, ossia il numero naturale viene fatto precedere da un ulteriore simbolo $+/-$. Nel sistema binario, abbiamo a disposizione unicamente 2 simboli. Inoltre in un computer il numero di cifre che compongono un numero e' prefissato. Il problema e' dunque come codificare questa informazione col numero di bit a disposizione.

7 Codifica

Dato il numero N di bit utilizzabili, consideriamo due modalita' di rappresentazione principali:

- Modulo e segno
- Complemento a 2

7.1 Rappresentazione in modulo e segno

Il bit piu' significativo indica il segno mentre gli altri bit rappresentano il numero in valore assoluto:

- $+$ ha codifica 0
- $-$ ha codifica 1

Esempio: $N = 8$

$$00000001- > +1 \quad (11)$$

$$1000\ 0001 -> -1$$

7.1.1 Intervallo di rappresentazione

$$[-2^{N-1} + 1, +2^{N-1} - 1] \quad (12)$$

Es: $N=8$:

$$[-2^7 + 1, +2^7 - 1] -> [-127, +127] \quad (13)$$

7.1.2 Svantaggi

Il problema e' che duplico la rappresentazione dello zero, spreco della rappresentazione ed inoltre complica le operazioni aritmetiche.

7.2 Rappresentazione in complemento a 2

L'idea e' di usare meta' delle possibili configurazioni (2^N) per rappresentare i numeri positivi e meta' i numeri negativi. I numeri positivi, compreso lo 0, hanno il bit piu' significativo a 0. Quelli negativi a 1. Esempio:

000	0
001	+1
010	+2
011	+3
111	-1
110	-2
101	-3
100	-4

Table 3: Esempio rappresentazione in Ca2

7.2.1 Intervallo di rappresentazione

$-2^{N-1}, +2^{N-1} - 1$ La codifica dello 0 e' unica, quindi l'intervallo di rappresentazione comprende un numero in piu' rispetto alla codifica in modulo e segno. Esempi: per N=3 [-4, +3], per N=4 [-8, +7], per N=8 [-128, +127].

7.3 Rappresentazione in Ca2 di un numero

Si consideri un numero x compreso nell'intervallo di rappresentazione $[-2^{N-1}, 2^{N-1} - 1]$. La rappresentazione di x_c in complemento a 2 di x si costruisce come segue:

- se $x \geq 0$ allora x_c coincide con la rappresentazione di x come intero SENZA segno
- se $x < 0$ x_c coincide con la rappresentazione (come intero SENZA segno) di $2^N - |x|$

Esempi N=4:

- $X_c = 0101 \rightarrow$ il segno e' +, quindi $x = +5$
- $X_c = 1110 \rightarrow$ il segno e' -, quindi $x = -(16 - 14) = -2$
- $X_c = 1001 \rightarrow$ il segno e' -, quindi $x = -(16 - 9) = -7$

Esempi N=8:

- $X_c = 01111111 \rightarrow x = +127$
- $X_c = 11110000 \rightarrow$ il segno e' negativo. Interpretato il numero come numero naturale: +240. Sottrazione: $2^8 - 240 = -16$

7.4 Regole pratiche per facilitare i calcoli

L'opposto di un numero x_c in complemento a due si ottiene ricopiando le stesse cifre da destra a sinistra fino al primo 1 e poi invertendo le rimanenti (0->1, 1->0). Oppure si effettua il complemento a uno del numero (si invertono tutte le cifre: 0->1, 1->0) quindi si somma 1.

7.5 Addizione/sottrazione in Ca2

- La somma di 2 numeri $x_c + y_c$ in complemento a due e' uguale al complemento a due della somma: $(x + y)_c = x_c + y_c$
- L'operazione di sottrazione tra due numeri interi puo' essere effettuata come addizione: $x_c - y_c = x_c + (-y)_c$

Queste proprieta' rendono piu' semplici i circuiti che realizzano le operazioni aritmetiche.

7.6 Overflow nella addizione in Ca2

Si ricorda che l'addizione genera overflow quando la somma non ricade nell'intervallo di rappresentazione, e quindi non e' rappresentabile. Nella pratica per determinare se si e' verificato overflow sappiamo che la somma fra un numero positivo e un numero negativo non genera mai overflow, al contrario si genera quando gli operandi sono entrambi positivi o negativi e il segno del risultato e' diverso da quello degli operandi.

8 Conversione da base 10 a base 2

Consideriamo un numero x,y . Riscriviamo il numero come: $x,y = x + 0,y$. Il metodo consiste nel convertire separatamente la parte intera e la parte frazionaria. Per la parte intera (x) si rappresenta il numero come intero senza segno. Per la parte frazionaria ($0,y$) si usa l'algoritmo illustrato in seguito.

8.1 Algoritmo

parte frazionaria $0,y$:

$i = 1$

Moltiplico $0,y$ per 2

Estrai la parte intera: questa rappresenta la cifra i -esima

$i = i + 1$

Estrai la parte frazionaria $\rightarrow 0,y$

Si prosegue fino a che $y = 0$

L'algoritmo di conversione della parte frazionaria produce a ogni passo una nuova cifra, la cifra prodotta e' la parte intera del risultato della moltiplicazione, l'ordine con cui vengono prodotte le cifre e' da sinistra verso destra ("allontanandosi" dalla virgola di separazione) da quella piu' significativa a quella meno significativa. Convertire 15,125

Resto	Intero
$15 : 2 = 7 \underline{1}$	$0,125 \times 2 = 0,25 \underline{0}$
$7 : 2 = 3 \underline{1}$	$0,25 \times 2 = 0,5 \underline{0}$
$3 : 2 = 1 \underline{1}$	$0,5 \times 2 = 1 \underline{1}$
$1 : 2 = 0 \underline{1}$	parte frazionaria=0 fine
Parte intera: <u>1111</u>	Parte frazionaria: <u>001</u>

Table 4: Esempio di conversione

8.2 Numero di cifre illimitato

Si possono ottenere numeri binari periodici. Un numero in base 10 con una parte frazionaria costituita da un numero finito di cifre puo' avere un numero di cifre infinito o molto grande in un'altra base.

8.3 Approssimazione di un numero

Con un numero finito di cifre e' solo possibile rappresentare un numero che approssima con un certo errore il numero reale dato. In pratica vuol dire che se facciamo conti con il computer in cui si usano numeri con la parte frazionaria, la componente errore dev'essere tenuta in considerazione.

8.4 Conversione da base 2 a base 10

Le cifre della parte frazionaria hanno pesi negativi. La n-esima cifra della parte frazionaria dopo la virgola ha peso 2^{-n} . Per ottenere il numero in base 10, si sommano i contributi di tutte le cifre, sia di quelle della parte frazionaria sia che della parte intera. Convertire $1101,11_2$:

- Parte intera: $1x2^3 + 1x2^2 + 0x2^1 + 1x2^0 = 13$
- Parte frazionaria: $1x2^{-1} + 1x2^{-2} = 1/2 + 1/4 = 0,5 + 0,25 = 0,75$
- $1001,11 \rightarrow 13,75$

8.5 Rappresentazione con N cifre

La questione ora è come separare la parte intera da quella frazionaria, dato un numero finito di cifre. Ci sono due tipi di codifica:

- Virgola fissa
- Virgola mobile: è quella comunemente usata per i numeri di tipo float

9 Virgola fissa

Si rappresenta il numero con N cifre, riservando un numero fisso di cifre per la parte intera e per la parte frazionaria, rispettivamente. Nell'esempio, si usano

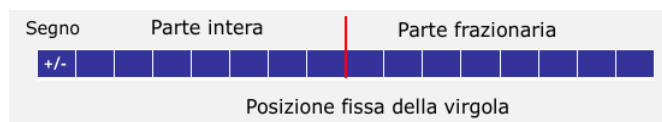


Figure 1: Virgola Fissa

8 cifre per la parte intera e segno, e 8 per la parte frazionaria. Il numero di cifre della parte frazionaria definisce la precisione della rappresentazione. Nell'esempio la precisione è 8.

9.1 Esempio

Si assuma $N=8$ con: 4 cifre per rappresentare la parte intera con segno, 4 cifre per la parte frazionaria. Rappresentare:

- $11,01 \rightarrow 0011|0100$
- $11101,01 \rightarrow$ Cifre insufficienti per la parte intera. Il numero non è rappresentabile.
- $11,10111 \rightarrow$ Si può solo rappresentare un'approssimazione del numero

- 0,00101111 -> Come sopra. Da notare che i bit riservati alla parte intera non sono usati.

Il limite della rappresentazione in virgola fissa e' che non si possono rappresentare numeri molto grandi o numeri molto piccoli.

10 Virgola mobile (floating point)

La codifica in virgola mobile corrisponde alla notazione scientifica usata nel sistema di numerazione decimale in base alla quale il numero e' espresso in termini di potenze di 10:

- 354,7 puo' essere espresso come $3,457 \times 10^2$
- 0,075 puo' essere espresso come $7,5 \times 10^{-2}$

In modo analogo, nel sistema di numerazione binario:

- 101,1 puo' essere espresso come $1,011 \times 2^2$

La moltiplicazione di un numero con parte frazionaria per una potenza della base con esponente positivo E equivale allo spostamento della virgola sulla destra di E posizioni. Se l'esponente e' negativo (-E), lo spostamento avviene a sinistra di E posizioni.

10.1 Forma normalizzata

E' un numero binario espresso nella forma:

$$+/- 1, Mx2^e \quad (14)$$

La cifra a sinistra della virgola e' 1.

- Mantissa: e' il valore posto a destra della virgola, quindi la parte frazionaria, il numero 1,M viene indicato col termine "significando". Il segno +/- si riferisce al significando.
- Esponente e: il valore dell'esponente della base 2

Esempi:

$$+0,01_2 \rightarrow 1,0x2^{-2}$$

$$+11_2 \rightarrow +1,1x2^1$$

$$+101,01_2 \rightarrow +1,0101x2^2$$

L'idea generale e' di suddividere le cifre a disposizione tra mantissa ed esponente, e di codificare le 2 componenti usando una qualche codifica.

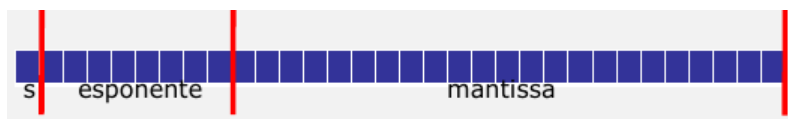


Figure 2: Virgola mobile in precisione singola: N=32

10.2 Standard IEEE-754

Standard utilizzato in ogni computer a partire dagli anni '80 per la rappresentazione dei numeri in virgola mobile

- S: segno del significando (+ -> 0, - -> 1)
- Mantissa: si rappresenta su 23 bit (solo la parte frazionaria del significando (1,xxxx))
- Esponente: su 8 bit incluso il segno dell'esponente

Si utilizza la codifica polarizzata a 127 (anche detta in eccesso 127) ovvero si rappresentano in numeri da -126 a +127, sommando ad ogni numero la quantità +127. Quindi il numero -126 è codificato come +1, mentre +127 corrisponde a +254. Esempio -> rappresentare -10,75:

1. Conversione a binario: $-10,65_{10} = -1010,11_2$
2. Normalizzazione: $-1,01011x2^3$
3. Codifica del segno: 1 = "-"; 0 = "+"
4. Calcolo dell'esponente in rappresentazione polarizzata: $e = 3 + 127 = 130_{10} = 10000010_2$
5. Risultato: $10,75_{10} = 1 \mid 1000\ 0010 \mid 01011000\ 00000000\ 00000000$

10.2.1 Configurazioni particolari

Numero	Mantissa	Esponente
0	=0...0	0000 0000
∞	=0...0	1111 1111
Nan (Not-a-Number)	$\neq 0...0$	1111 1111
Num. denormalizzato	$\neq 0...0$	0000 0000

Table 5: Configurazioni speciali

11 Rappresentazione dei caratteri

Il carattere e' la piu' piccola unita' di testo rappresentabile. Concetti alla base della rappresentazione dei caratteri:

- **Repertorio** dei caratteri: l'insieme dei caratteri. Esempio: l'insieme dei caratteri latini
- **Codice**: corrispondenza uno-a-uno fra caratteri e numeri (codici numerici). Il codice e' comunemente rappresentato in forma tabulare
- **Codifica**: rappresentazione binaria dei codici numerici

11.1 Standard ASCII

Gli standard sono importanti perche' definiscono delle norme comuni a cui bisogna attenersi per garantire interoperabilita' (i dati su un computer possono essere portati anche su un computer diverso ed essere decodificati in modo corretto). ASCII - American Standard Code for Information Interchange, e' la codifica piu' nota nel mondo dell'informatica. E' standard ANSI (American National Standard Institute) dal 1968 ed e' costituito da 128 caratteri. Si riferisce contemporaneamente ad un repertorio di caratteri e alla loro codifica. Il carattere e' codificato su 7 bit. Ogni carattere occupa 1 byte.

11.1.1 Repertorio

- 33 caratteri sono caratteri di controllo. Esempio ESC, CR. Derivano tendenzialmente dall'epoca dei telegrammi.
- 95 caratteri sono composti dai caratteri dell'alfabeto latino, minuscole, maiuscole, numeri e punteggiatura

11.1.2 Codice

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	NUL	32	20	SPACE	64	40	@	96	60	–
1	1	(START OF HEADING)	33	21	!	65	41	A	97	61	a
2	2	(START OF TEXT)	34	22	"	66	42	B	98	62	b
3	3	(END OF TEXT)	35	23	#	67	43	C	99	63	c
4	4	(END OF TRANSMISSION)	36	24	\$	68	44	D	100	64	d
5	5	(ENQUIRY)	37	25	%	69	45	E	101	65	e
6	6	(ACKNOWLEDGE)	38	26	&	70	46	F	102	66	f
7	7	(BEL)	39	27	'	71	47	G	103	67	g
8	8	(BACKSPACE)	40	28	(	72	48	H	104	68	h
9	9	(HORIZONTAL TAB)	41	29	)	73	49	I	105	69	i
10	A	(LINE FEED)	42	2A	+	74	4A	J	106	6A	j
11	B	(VERTICAL TAB)	43	2B	=	75	4B	K	107	6B	k
12	C	(FORM FEED)	44	2C	,	76	4C	L	108	6C	l
13	D	(CARriage RETURN)	45	2D	-	77	4D	M	109	6D	m
14	E	(SHIFT OUT)	46	2E	.	78	4E	N	110	6E	n
15	F	(SHIFT IN)	47	2F	/	79	4F	O	111	6F	o
16	10	(DATA LINK ESCAPE)	48	30	0	80	50	P	112	70	p
17	11	(DEVICE CONTROL 1)	49	31	1	81	51	Q	113	71	q
18	12	(DEVICE CONTROL 2)	50	32	2	82	52	R	114	72	r
19	13	(DEVICE CONTROL 3)	51	33	3	83	53	S	115	73	s
20	14	(DEVICE CONTROL 4)	52	34	4	84	54	T	116	74	t
21	15	(NEGATIVE ACKNOWLEDGE)	53	35	5	85	55	U	117	75	u
22	16	(SYNCHRONOUS GATE)	54	36	6	86	56	V	118	76	v
23	17	(END OF TRANSMISSION)	55	37	7	87	57	W	119	77	w
24	18	(CANCEL)	56	38	8	88	58	X	120	78	x
25	19	(END OF MEDIUM)	57	39	9	89	59	Y	121	79	y
26	1A	(SUBSTITUTE)	58	3A	:	90	5A	Z	122	7A	z
27	1B	(ESC)	59	3B	;	91	5B	[	123	7B	{
28	1C	(FILE SEPARATOR)	60	3C	<	92	5C	\	124	7C	
29	1D	(GROUP SEPARATOR)	61	3D	=	93	5D	]	125	7D	}
30	1E	(RECORD SEPARATOR)	62	3E	>	94	5E	^	126	7E	~
31	1F	(UNIT SEPARATOR)	63	3F	?	95	5F	_	127	7F	[DEL]

Figure 3: ASCII table

12 La famiglia ISO 8859

Estende il repertorio di caratteri dell'ASCII introducendo caratteri speciali a seconda della lingua usata. La codifica e' su 8 bit, quindi si estende il repertorio a 256 caratteri. Si tratta di una famiglia di codifiche che comprende, ad esempio, l'ISO 8859-1 (latin-1) che contiene i caratteri principali delle lingue occidentali con alfabeti latini ed e' compatibile con ASCII. Un'altra codifica e' 8859-2(ISO Latin2) che contiene i caratteri per le lingue dell'Europa Centrale e dell Est. Il problema e' che il numero di simboli sono ancora limitati per quelle che sono le esigenze pressanti, perche' abbiamo l'esigenza di rappresentare altri simboli (emoticon).

12.1 Unicode

Sistema per la codifica dei caratteri proposto come standard nel 1991. Sistema realizzato da un consorzio di aziende che include tutte le societa' leader del periodo. Attualmente lo standard e' mantenuto da un consorzio. La codifica Unicode e' ampiamente utilizzata in linguaggi, sistemi operativi ed altre operazioni. Era originalmente progettato per essere un codice a 16 bit. Successivamente fu esteso in modo da permettere codici nell' intervallo esadecimale 0...10FFFF vale a dire 1114112 caratteri. Tipicamente un carattere Unicode viene identificato con l'abbreviazione U+xxxx dove xxxx e' un numero esadecimane a quattro cifre (fino a 6 cifre per l'estensione).

12.2 Codifica

Esistono molteplici codifiche per il codice Unicode. La codifica e' identificata col termine Unicode Trnformation Format (UTF).

- UTF-8: impiega 1 byte per i caratteri ASCII,
- UTF-16: usa 2 byte

13 Introduzione

I circuiti digitali (digit=cifra) sono i dispositivi che permettono la elaborazione dei dati in forma binaria. I circuiti digitali elementari sono le **porte logiche**. Circuiti complessi sono realizzati come insiemi di porte logiche fra loro connesse (reti logiche). Il nome di porta logica deriva dal fatto che questi circuiti elementari implementano funzioni logiche. Le funzioni logiche sono definite in termini matematici dall'algebra di Bool (o algebra booleana).

13.1 Brevi note storiche

George Boole, matematico (1825-1864), crea il legame fra logica e matematica: gli enunciati logici sono espressi mediante operazioni algebriche. Claude Shannon, ingegnere e matematico (1916-2001) dimostra che la algebra di Boole puo' essere utilizzata per descrivere il funzionamento dei circuiti.

14 Circuito digitale

Un circuito digitale lo possiamo vedere come una scatola nera (si intende la cui struttura interna puo' non essere rilevante, ma puo' interagire con altri componenti) che presenta delle linee in ingresso e delle linee in uscita. Attraverso queste linee si propaga un segnale elettrico che puo' assumere due valori distinti, convenzionalmente rappresentati dal valore 0 e 1, quindi da bit. I valori dei segnali in ingresso possono variare nel tempo e di conseguenza anche i valori in uscita

14.1 Circuiti combinatori

Il circuito puo' essere descritto in termini matematici da una funzione, detta funzione logica. La funzione definisce la corrispondenza tra valori in ingresso e valori di uscita. E' detto circuito senza memoria. Per semplicita', assumiamo che ci sia una sola linea di uscita:

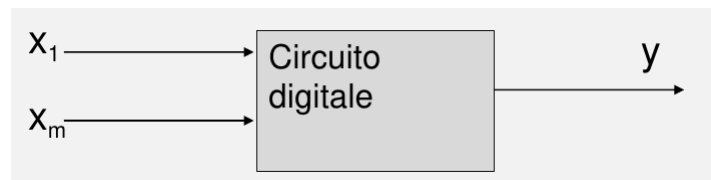


Figure 4: Modello circuito digitale combinatorio ad un'uscita

Il circuito puo' essere descritto in termini matematici da una funzione, detta funzione logica. La funzione definisce la corrispondenza tra valori di ingresso e valori di uscita:

$$y = f(x_1, \dots, x_m) \quad (15)$$

y, $X_1, \dots, X_m$ sono variabili binarie: $X_1, \dots, X_m$ sono le variabili in ingresso, y la variabile in uscita. In un circuito combinatorio il valore in uscita dipende unicamente dal valore in ingresso.

14.2 Tabella di verita' di una funzione booleana

La tabella di verita' specifica il valore della funzione booleana per ogni possibile combinazione dei valori in ingresso. In particolare, la tabella di verita' per una funzione a n ingressi ha 2^n righe, che corrispondono a tutte le possibili configurazioni dei valori, e due "gruppi" di colonne:

- colonne degli ingressi, una per ogni variabile in ingresso
- colonna dell'uscita, per la variabile in uscita

x_1	X_2	$f(X_1, X_2)$
0	0	...
0	1	...
1	0	...
1	1	...

Table 6: Tabella di verita' per una funzione f a 2 variabili

La tabella di verita' e' univoca per ogni funzione. Si riportano tutte le combinazioni possibili delle variabili, mantenendo l'ordine. Una funzione booleana e logica e' definita da una tabella di verita', la quale e' unica.

15 Algebra di Boole

Possiamo anche definire la funzione usando un'espressione dell'algebra booleana. Algebra booleana:

- Costanti binarie: Falso(=0); VERO(=1)
- Variabili binaria: e.g. A,B,C,a,b,c...
- Operatori logici: NOT, AND, OR
- Proprieta' degli operatori logici definite da postulati e teoremi

L'espressione booleana e' composta da operatori logici, costanti binarie, variabili binarie, parentesi. Una funzione logica si scrive: variabile binaria = espressione booleana, $Y = \text{espressione booleana}$.

15.1 Operatori logici

- A AND B: Si scrive in modo sintetico AB. Operazione di prodotto logico. Il risultato dell'operazione e' vero (1) se e solo se A e B sono entrambi uguali a 1 altrimenti 0.
- A OR B: si scrive $Y=A+B$. Operazione di somma logica. Il risultato dell'operazione e' falso (0) se e solo se A e B sono entrambi uguali a 0.
- NOT A: Si scrive con un tratto sopra la variabile (si legge: A negato). Operazione di negazione. L'operazione e' anche detta di complementazione, perche' trasforma una variabile nel suo complemento (0->1, 1->0)

A	B	Y=AB
0	0	0
0	1	0
1	0	0
1	1	1

Table 7: Tabelli di verita' AND

A	B	Y=AB
0	0	0
0	1	1
1	0	1
1	1	1

Table 8: Tabelli di verita' OR

A	Y=NOT A
0	1
1	0

Table 9: Tabelli di verita' NOT

15.2 Espressioni booleane

L'espressione booleana viene composta utilizzando costanti, variabili, operatori e parentesi.

15.2.1 Regole di precedenza degli operatori

In assenza di parentesi, le operazioni devono essere effettuate nel seguente ordine:

- AND ha priorit  sull'OR
- NOT ha priorit  su AND e OR

15.3 Dall'espressione alla tabella di verita' di una funzione

Data una funzione booleana espressa in forma algebrica (tramite l'uso di operatori logici), come si costruisce la tabella di verita' della funzione? La tabella di verita' si crea elencando prima tutti i possibili valori delle variabili di ingresso e poi calcolando il valore della funzione per ogni configurazione. Per semplificare si possono aggiungere delle ulteriori colonne che contengono il risultato delle sotto-espressioni. In questo modo la funzione viene costruita per passi successivi.

x	y	yx	x	z
0	0	0	1	1
0	1	0	1	1
1	0	0	0	0
1	1	1	0	1

Table 10: $z = \neg x + yx$

15.4 Proprieta' degli operatori

Le proprieta' degli operatori sono espresse tramite identita' logiche. Data un'identita' logica, se ne puo' ottenere una duale (diversa dalla precedente) scambiando fra loro gli operatori AND e OR e scambiando fra loro gli 1 e gli 0.

$$A \cdot A = 0 \rightarrow A + \neg A = 1 \quad (16)$$

Le proprieta': commutativa, distributiva, identita', elemento inverso sono postulati: assunte vere per definizione. Le altre proprieta' sono teoremi che si ottengono per dimostrazione a partire dai postulati. Le identita' elencate continuano a valere se si sostituiscono alle variabili espressioni booleane arbitrariamente complesse.

	And	Or (duale)
Identita'	$1 \cdot x = x$	$0 + x = x$
Elemento zero	$0 \cdot x = 0$	$1 + x = 1$
Idempotenza	$x \cdot \neg x = 0$	$x + \neg x = 1$
Commutativa	$x \cdot y = y \cdot x$	$x + y = y + x$
Associativa	$(x \cdot y) \cdot z = x \cdot (y \cdot z) = xyz$	$(x + y) + z = x + (y + z) = x + y + z$
Distributiva	$x \cdot (y + z) = x \cdot y + x \cdot z$	$x + (y \cdot z) = (x + y) \cdot (x + z)$
I assorbimento	$x \cdot (x + y) = x$	$x + x \cdot y = x$
II assorbimento	$x \cdot (\neg x + y) = x \cdot y$	$x + \neg x \cdot y = x + y$
De Morgan	$\neg(x \cdot y) = \neg x + \neg y$	$\neg(x + y) = \neg x \cdot \neg y$

Table 11: Proprieta'

Esempi :

- $ABC + 1 = 1$
- $ABC + ABC = ABC$
- $ABC \cdot 0 = 0$
- $ABC \cdot ABC = ABC$
- $AB(C + A) = ABC + ABA = ABC + AAB = ABC + AB = AB(C + 1) = AB(1) = AB$
- $A + \neg A \cdot B = A + B$
- $\neg ABC = \neg A + \neg B + \neg C$

15.5 Espressioni equivalenti

Sono **equivalenti** due espressioni e_1, e_2 con le stesse variabili e tali che $e_1 = e_2$ per tutte le assegnazioni delle variabili. Quindi le due espressioni descrivono la stessa funzione booleana e pertanto hanno la stessa tabella di verità. **Semplificare** una espressione booleana significa ricondurre l'espressione ad una espressione equivalente più semplice. Per semplificare in modo manuale un'espressione booleana si applicano le identità dell'algebra booleana. La semplificazione algebrica è importante perché permette di semplificare i circuiti.

15.5.1 Esempio

Semplificare $Y + AB\neg C + AC$, applichiamo le identità algebriche, quindi verifichiamo la correttezza:

- $AB\neg C + AC = A(B\neg C + C)$ distributiva
- $= A(B + C)$ II assorbimento
- $AB + AC$

A	B	C	$\neg C$	$AB\neg C$	AC	Y
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	1	1
1	1	0	1	1	0	1
1	1	1	0	0	1	1

Table 12: Tabella di verità

15.6 Operatori funzionalmente completi

La terna di operatori AND, OR, NOT sono **funzionalmente completi** perche' permettono di rappresentare ogni funzione booleana. Non e' l'unico insieme:

- Operatore NAND e' funzionalmente completo : $ANANDB = \neg AB$
- Operatore NOR e' funzionalmente completo: $ANORB = \neg A + B$

15.7 Forme Canoniche

Una funzione booleana puo' essere espressa in forme algebriche tra loro equivalenti. Alcune di queste sono chiamate forme canoniche. Consideriamo solo la prima forma canonica o forma canonica di somma di prodotti (SOP).

15.7.1 Prima forma canonica

Si consideri la tabella di verita' di una funzione booleana. Si vuole ricavare la forma canonica SOP. Si definisce **mintermine** un prodotto nel quale appaiono tutte le variabili della funzione in forma diretta o inversa (negata), dato $F(x_1, x_2, x_3)$ sono mintermini:

- $x_1 \cdot x_2 \cdot x_3$
- $\neg x_1 \cdot x_2 \cdot x_3$
- $x_1 \cdot \neg x_2 \cdot \neg x_3$

Con n variabili, vi sono 2^n mintermini. **Prima forma canonica SOP:** data una tabella di verita', la funzione e' costruita come somma logica dei mintermini che fanno assumere valore 1 alla funzione, dove le variabili compaiono in forma diretta se il valore della variabile e' 1 e in forma negata se il valore e' 0.

15.7.2 Esempio

$$f = \neg A \cdot B \cdot \neg C + A \cdot B \cdot \neg C + A \cdot B \cdot C$$

A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Table 13: Tabella di verita'

Mintermini: il prodotto delle variabili e' 1 se e solo se $f = 1$. Si crea un mintermine per ogni valore 1 della funzione. Le variabili nel mintermine sono presenti in forma diretta se il valore nella tabella e' 1 e negata se il valore e' 0.

15.8 Porte logiche e simboli funzionali

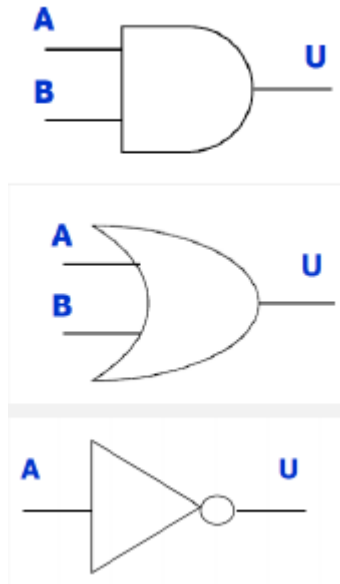


Figure 5: AND, OR, NOT

Le porte logiche AND e OR possono avere piu' di 2 ingressi

15.9 Porte universali NAND e NOR

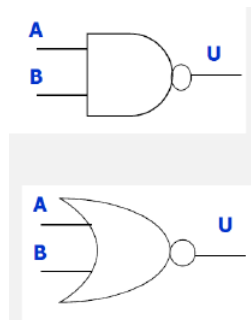


Figure 6: NAND e NOR a due ingressi

15.10 Da funzione in forma algebrica al circuito

Data la funzione $F = AB + B\bar{C}$, costruire il circuito che la implementa:

- Variabili in ingresso: A, B, C
- Variabile in uscita: F
- Porte logiche:
 - 2 porte AND
 - 1 porta OR
 - 1 porta NOT

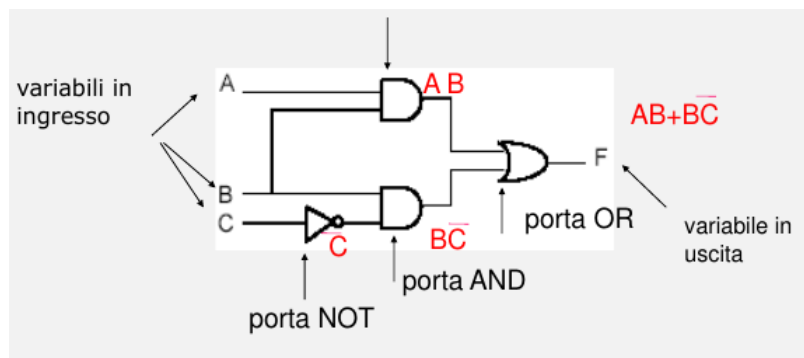


Figure 7: $F = AB + B\bar{C}$

15.11 Convenzioni grafiche

Per disegnare un circuito:

- Ingressi sulla sinistra, output sulla destra
- Se possibile le porte si sviluppano dalla sinistra alla destra (o dall'alto verso il basso)
- Linee diritte possibilmente. Linee possono essere connesse, un punto indica la connessione

15.12 Analisi di un circuito combinatorio

Dato un circuito, comprendere il suo funzionamento tramite la specifica della funzione logica che il circuito implementa. La funzione può essere espressa in forma algebrica oppure tramite tabella di verità.

15.12.1 Esempio1

Considerare il seguente circuito ed esprimere la funzione logica in forma algebrica:

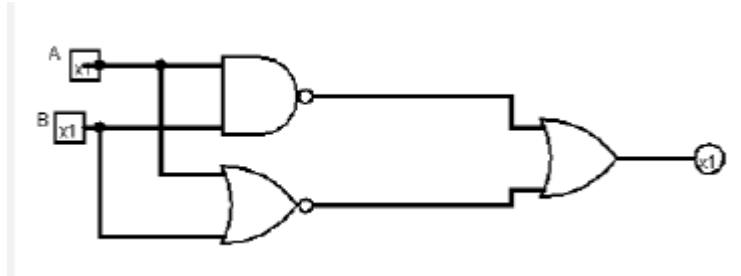


Figure 8: Esempio 1

Comporre l'espressione osservando come si propagano i segnali in ingresso attraverso le porte.

$$Y = ANANDB + ANORB = \neg AB + \neg A + B \quad (17)$$

15.12.2 Esempio2

Considerare il seguente circuito ed esprimere la funzione logica tramite tabella di verita':

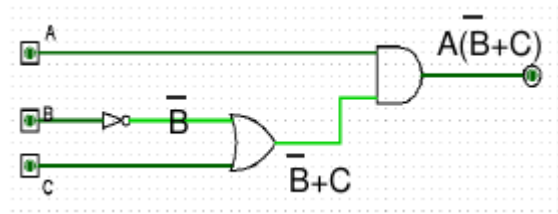


Figure 9: Esempio 2

$$F = A(\neg B + C) \quad (18)$$

15.13 Sintesi di un circuito combinatorio

Costruzione del circuito a partire dal problema descritto informalmente. Passi:

- Analisi e comprensione del problema
- Definizione della tabella di verita'

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Table 14: Tabella di verita'

- Dalla tabella di verita' alla espressione algebrica in forma canonica
- Eventuale semplificazione della funzione per ridurre la complessita' del circuito
- Disegno del circuito

15.13.1 Esempio : passo 1

Problema: ci sono 3 votanti che possono votare 0 e 1. Costruire il circuito che restituisce 1 se la maggioranza dei votanti ha votato 1. I 3 votanti li indico con le variabili A, B, C. Il valore della variabile in uscita Y indica il voto: $Y = f(A, B, C)$.

15.13.2 Esempio: passo 2

Costruzione della tabella di verita'.

A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Table 15: Tabella di verita'

15.13.3 Esempio: passo 3

Esprimere la funzione in forma canonica. I mintermini li troviamo nella tabella di verita', sono le righe in cui il valore di $f = 1$.

$$f(A, B, C) = \neg A \cdot B \cdot C + A \cdot \neg B \cdot C + A \cdot B \cdot \neg C + A \cdot B \cdot C \quad (19)$$

15.13.4 Esempio: passo 4

Disegno del circuito:

- 4 porte AND a 3 ingressi, una per ogni mintermine
- 1 porta OR a 4 ingressi (non controlliamo gli invertitori)
- Numero totale di porte: 5; numero totale di ingressi alle porte: 16

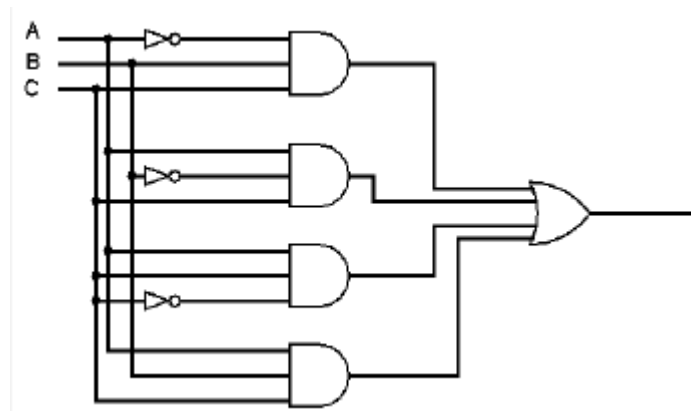


Figure 10: Circuito

15.14 Semplificazione del circuito

La complessita' di un circuito e' connessa alla complessita' della funzione logica. E' importante ridurre la complessita' del circuito per ridurre i costi e aumentare l'efficienza. Riducendo il numero di porte e il numero di ingressi alle porte, si ottiene un circuito piu' semplice. Riducendo il numero di porte attraversate dal segnale in ingresso si aumenta l'efficienza.

15.15 Propagazione del segnale: aspetti fisici

Le porte logiche operano sui segnali elettrici in ingresso per produrre un segnale d'uscita. I segnali elettrici (ad esempio, la tensione), assumono nel sistema 2 valori distinti e riconoscibili: **livello alto** = 1, **livello basso** = 0. I segnali sono continui nel tempo e possono commutare.

- Ritardo di propagazione di una porta logica (o tempo di commutazione): il tempo T necessario perche' una variazione degli ingressi si rifletta all'uscita. Dato una rete logica, il tempo di propagazione del segnale nel circuito risulta dalla somma dei tempi di propagazione attraverso le porte.
- Percorso critico: numero massimo di porte che vengono attraversate da un segnale dall'ingresso all'uscita. Ridurre il cammino critico aumenta l'efficienza.

16 Circuiti combinatori con uscita multipla

Finora abbiamo considerato funzioni booleane con una sola variabile in uscita. In generale, una funzione può avere più di una variabile in uscita. Una generica funzione ha forma:

$$F(X_1, \dots, X_n) = (Y_1, \dots, Y_m) \quad n, m \geq 1 \quad (20)$$

Per realizzare il circuito si costruiscono m circuiti a singola uscita che implementano le funzioni $F_1, F_2, \dots, F_m$: $F_1(x_1 \dots x_n) = y_1$, $F_2(x_1 \dots x_n) = y_2$. La tabella di verità del circuito si costruisce riportando in uscita una colonna per ogni variabile di uscita.

16.1 Esempio

Realizzare il circuito a 3 variabili in ingresso e 2 variabili in uscita: $F(A, B, C) = (D, E)$ dove $D=1$ se i valori in ingresso comprendono esattamente due 1, $E=1$ se i valori in ingresso sono tutti 1.

A	B	C	D	E
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

Table 16: Tabella di verità

$$D = \neg ABC + A\neg BC + AB\neg C \quad (21)$$

$$E = ABC \quad (22)$$

17 Blocchi funzionali combinatori

Una rete combinatoria complessa può essere realizzata combinando tra loro piccole reti realizzate come blocchi funzionali combinatori. L'idea è di avere una "libreria" di circuiti che realizzano funzioni comuni e poi comporre questi circuiti per realizzare funzioni complesse. Consideriamo i seguenti blocchi funzionali combinatori:

- Comparatore di uguaglianza: devo confrontare se due sequenze di bit sono identiche

- Decoder
- Multiplexer
- Sommatore

17.1 XOR

La funzione XOR (a,b) indica l'OR esclusivo: vale 1 se i valori delle variabili a e b sono diversi. A differenza della somma logica, se a e b sono entrambi a 1 (veri), la funzione XOR è falsa.

a	b	xor(a,b)
0	0	0
0	1	1
1	0	1
1	1	0

Table 17: Tabella di verità XOR

$$xor(a, b) = \neg a \cdot b + a \cdot \neg b = a \oplus b \quad (23)$$

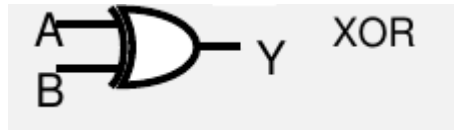


Figure 11: Simbolo della porta logica XOR

17.2 XNOR

La negazione della funzione XOR è la funzione XNOR, che vale 1 se a e b sono uguali. L'operatore in forma algebrica si indica come XOR negato.

a	b	xnor(a,b)
0	0	1
0	1	0
1	0	0
1	1	1

Table 18: Tabella di verità XNOR

$$xnor(a, b) = not(xor(a, b)) = \neg a \cdot \neg b + a \cdot b = \neg(a \oplus b) \quad (24)$$

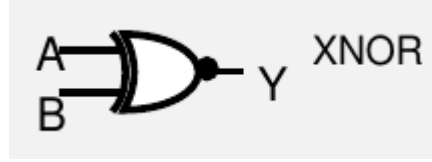


Figure 12: Simbolo della porta logica XNOR

17.3 Espressioni algebriche

Le espressioni algebriche si risolvono togliendo il simbolo, ovvero esplicitando secondo la definizione.

$$y = \neg(a \oplus b) \cdot c + (a \oplus b)y = (\neg a \neg b + ba)c + (\neg ab + \neg ba) = \neg a \neg bc + abc + \neg ab + \neg ba = \neg a(\neg bc + b) + a(bc + \neg b) = \neg a(b + \neg c) + a(b + c) = \neg a \neg c + a(b + c) \quad (25)$$

17.4 Comparatore di uguaglianza

Il comparatore confronta 2 sequenze di n bit. Le sequenze devono avere la stessa lunghezza, determina se 2 sequenze binarie sono uguali. Ha in ingresso 2 gruppi di n bit e in uscita 1 bit. Il valore in uscita e' 1 se A e B sono uguali, 0 altrimenti.

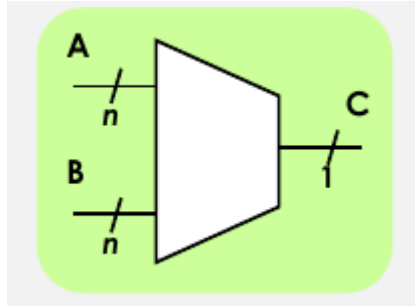


Figure 13: Simbolo del comparatore. Ogni parola di n bit viene indicata con una sola linea e il numero di bit

Il confronto viene effettuato bit a bit, utilizzando la porta XNOR per il confronto della singola coppia di bit. Significa che avro' bisogno di tante porte XNOR quante sono i bit della mia sequenza.

17.5 Decoder N-M

Ha generalmente N ingressi e 2^N uscite. Permette di specificare quale linea in uscita e' uguale a 1. Gli ingressi $I_{N-1} \dots I_0$ sono interpretati come numero intero (range di valori $[0, 2^{n-1}]$), mentre le uscite hanno un indice da O_0 a O_{2^N-1} . Se sugli ingressi e' presente il numero binario k, la k-esima uscita di

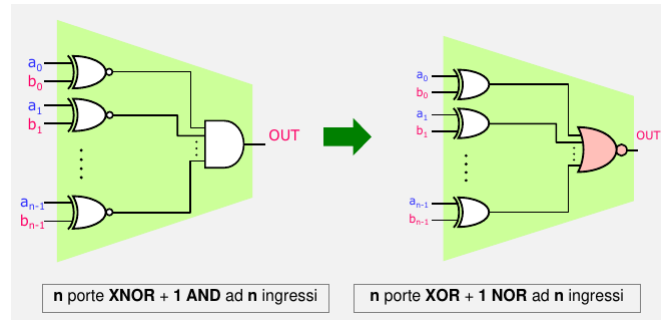


Figure 14: 2 schemi alternativi

O_k assume il valore 1 e le restanti uscite assumono il valore 0. E' utilizzato ad esempio nelle memorie, per selezionare una cella mediante il suo indirizzo. In sostanza il numero in ingresso sta ad identificare il numero della linea che vogliamo abilitare.

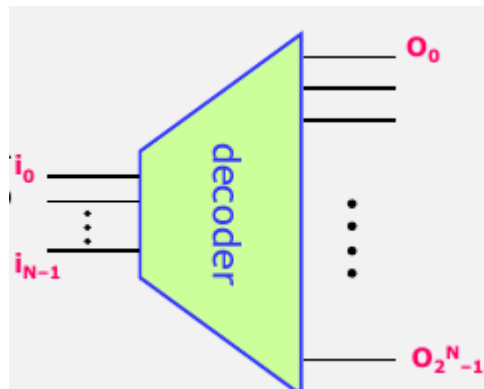


Figure 15: Decoder

17.5.1 Decoder 2-4

Associamo ad ogni bit in ingresso una variabile: I_1, I_0 . Il bit piu' significativo e' I_1 indichiamo le 4 variabili in uscita con U_0, U_1, U_2, U_3

Decoder piu' grandi possono essere costruiti seguendo la stessa tecnica impiegando una porta AND per ogni uscita, ciascuna con tanti ingressi quanti sono gli ingressi del decoder.

17.6 Multiplexer MUX

Il multiplexer e' un circuito che seleziona una delle linee di ingresso e la dirige a una singola linea di uscita. E' chiamato anche selettore. Ingressi:

I_1	I_0	U_0	U_1	U_2	U_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Table 19: Tabella di verita' Decoder 2-4

- n linee di input (considerare sempre le linee come se fossero bit): $I_0 \dots I_{n-1}$
- k linee di controllo: $S_0 \dots S_{k-1}$

Output: 1 linea. Il valore fornito sulla linea di controllo seleziona la linea di ingresso. Quante linee di selezione? $k = \text{ceiling}(\log_2 n)$

17.6.1 Multiplexer a 2 ingressi

Input: 2 ingressi x_0, x_1 + 1 linea di controllo S. Output: if $S=0$ then x_0 else x_1

S	X_0	X_1	U
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Table 20: Tabella di verita'

17.6.2 Multiplexer a piu' vie

MUX a n ingressi: si utilizza un DECODER con $k = \log_2(n)$ ingressi, n uscite. Il decoder ha in ingresso le linee di controllo del multiplexer, S. Le uscite del decoder servono a selezionare l'ingresso del multiplexer. La selezione della linea I_j viene realizzata tramite una porta AND tra l'uscita del decoder U_j e l'ingresso I_j . In questo modo, l'uscita del decoder abilita il passaggio del valore di ingresso. Un solo ingresso viene selezionato.

17.7 Circuito sommatore

Circuiti che permettono di calcolare la somma fra due numeri. Un sommatore effettua la somma bit per bit. Il sommatore risulta dalla composizione di circuiti che effettuano la somma tra 2 bit. Questi circuiti si distinguono in:

- Half-adder (mezzo sommatore)

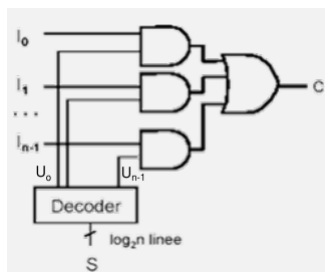


Figure 16: Multiplexer a piu' vie

- Full-adder (sommatore completo)

17.7.1 Half Adder

Somma aritmetica tra 2 bit (1 bit + 1 bit)

- 2 ingressi : addend: a, b
- 2 uscite: somma 2, riporto r

a	b	somma	riporto
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 21: Tabella di verita' SOMMA

$$s = a \oplus b \quad r = ab \quad (26)$$

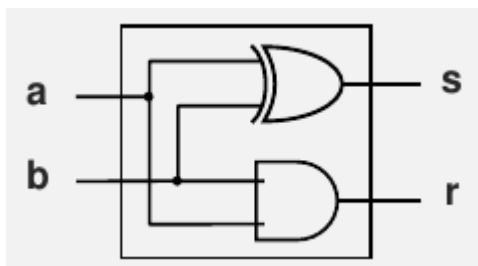


Figure 17: Half Adder

17.7.2 Full Adder

Somma fra due bit tenendo conto del riporto che puo' essere stato generato in precedenza

- 3 ingressi: a , b , r_{in}
- 2 uscite: s , r_{out}

a	b	r_{in}	r_{out}	s
0	0	0	0	0
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0
0	0	1	0	1
0	1	1	1	0
1	0	1	1	0
1	1	1	1	1

Table 22: Tabella di verita'

Prima forma canonica (SOP)

$$s = \overline{a}b\overline{r_{in}} + \overline{a}br_{in} + a\overline{b}\overline{r_{in}} + ab\overline{r_{in}}$$

$$r_{out} = a\overline{b}r_{in} + \overline{a}br_{in} + \overline{a}\overline{b}r_{in} + ab r_{in}$$

Semplifico (salto i passaggi):

$$s = (a \oplus b)\overline{r_{in}} + \overline{(a \oplus b)}r_{in} = (a \oplus b) \oplus r_{in}$$

$$r_{out} = ab + (a \oplus b)r_{in}$$

Figure 18: SOP full adder

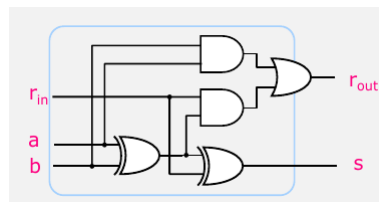


Figure 19: Circuito sommatore Full Adder

17.8 Sommatore di parole di N bit

Sommatore a **propagazione di riporto**. Consideriamo 2 numeri a n bit. Il sommatore a propagazione di riporto e' costituito da n-1 circuiti FA e 1 circuito HA. Ogni circuito effettua la somma di 2 bit e propaga il riporto.

- Input: 2 parole di N bit
- Output: somma di N+1 bit

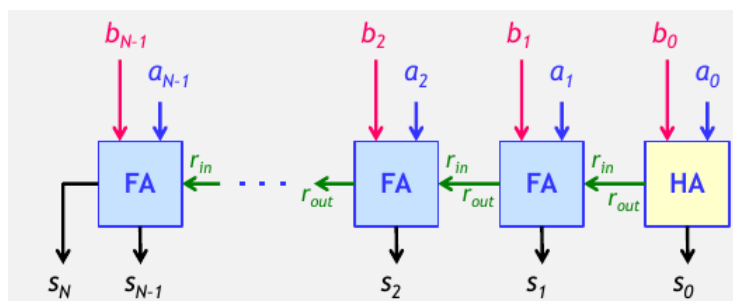


Figure 20: Sommatore di parole di N bit

18 Introduzione

I **circuiti combinatori** sono **circuiti senza memoria**, possono essere descritti da una funzione, ci sono piu' ingressi ed una o piu' uscite. Ogni uscita e' una funzione logica degli ingressi. Sono caratterizzati dall'indipendenza dal tempo e appunto la mancanza di memoria. Al contrario i **circuiti sequenziali** sono circuiti **con memoria**, la memoria contiene lo stato (X) del sistema.

19 Circuiti sequenziali

Un circuito sequenziale e' costituito da un circuito combinatorio e da elementi di memoria interconnessi in un anello. **Elementi di memoria** sono circuiti in grado di immagazzinare informazioni binarie. L'uscita del circuito e' funzione degli ingressi e dello stato corrente. Analogamente lo stato futuro

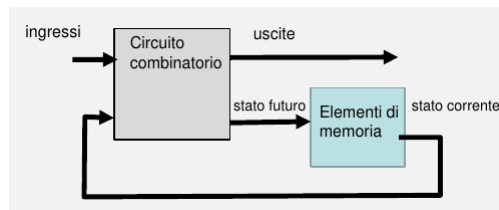


Figure 21: Circuito sequenziale

- **Circuito sequenziale sincrono:** il comportamento del circuito evolve nel tempo in modo discreto. La elaborazione e propagazione dei segnali e' scandita da un orologio comune (clock).
- **Circuito sequenziale asincrono:** gli ingressi evolvono in modo continuo e di conseguenza anche il comportamento del circuito.

La progettazione di circuiti asincroni e' piu' complessa, perche' il comportamento dipende dal tempo di propagazione delle porte e dall'istante in cui gli ingressi cambiano. Tendono ad essere preferiti i circuiti sincroni.

19.1 Evoluzione temporale del segnale

Un segnale varia nel tempo. Il diagramma temporale e' un grafico che descrive l'evoluzione temporale del segnale che puo' commutare da alto a basso e viceversa.

La commutazione del segnale in uscita a fronte di una commutazione dei segnali in ingresso non e' immediata. Il tempo necessario perche' una variazione degli ingressi si rifletta all'uscita costituisce il ritardo di propagazione.

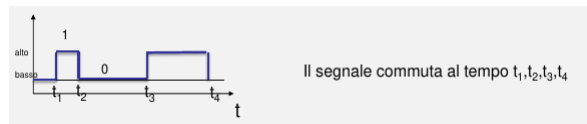


Figure 22: Diagramma temporale

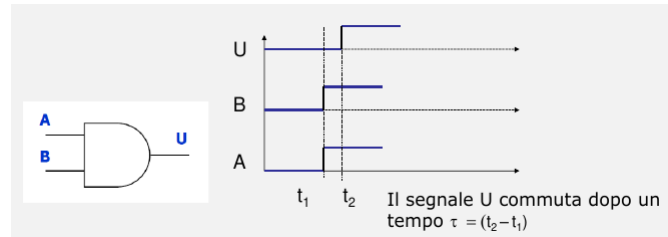


Figure 23: Ritardo di propagazione

19.2 Retroazione

Sono circuiti in cui l'uscita diventa a sua volta ingresso formando un anello. Se t e' il ritardo di propagazione, il segnale in uscita e' stabile per un intervallo di durata t , poi il segnale si inverte.

20 Elemento di memoria

Nei sistemi digitali le informazioni vengono immagazzinate in modi diversi inclusi tramite circuiti. L'elemento di memoria e' un circuito che e' in grado di memorizzare 1 bit, pureche' alimentato, fino a quando l'informazione non viene deliberatamente modificata. Il circuito puo' essere asincrono o sincrono.

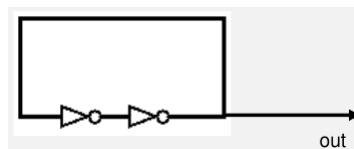


Figure 24: Prima idea di elemento di memoria

$OUT = 1$ (out=0) per un tempo limitato. Il circuito e' in grado di immagazzinare un bit sfruttando il ritardo di propagazione e il meccanismo di retroazione, tuttavia NON e' possibile modificare il dato se non modificando il circuito.

21 Circuito Latch di tipo SR

Circuiti di memoria di tipo asincrono alla base di circuiti piu' cpmplexi. Consideriamo il circuito costituito da 2 porte NOR con collegamenti incrociati, 2 ingressi indicati come S (set) e R (reset), da un'uscita Q e dala sua negazione $\bar{Q}$.

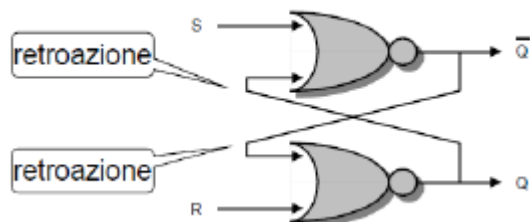


Figure 25: Circuito Latch SR

Il circuito si puo' trovare in due stati di funzionamento: stato di set per cui l'uscite e' $Q=1$ o stato di rest per cui l'uscita e' $Q=0$. Il valore di Q indica il valore immagazzinato presente in uscite. Se entrambi gli ingressi R e S sono a 0, lo stato di funzionamento non cambia.

$R=0, S=0, Q=0$		L'uscita non si modifica $Q=0$
$R=0, S=0, Q=1$		L'uscita non si modifica $Q=1$

Table 23:

Quando vi e' una commutazione dello stato di funzionamento il circuito si stabilizza dopo $2t$, dove t e' il ritardo di commutazione.

21.1 Stato indefinito

Se i segnali in ingresso si portano contemporaneamente a 1, entrambe le uscite Q e $\bar{Q}$ sono a 0 contro l'ipotesi di funzionamento normale secondo cui le uscite sono l'unil complementodell'altra. *IL se Re S sono portati di nuovo entrambi a 0, none' possibile prevedere l'uscita perche Q e Q continuano ad oscillare fra 00*

21.2 Segnali anomali

Si crea un segnale anomalo a causa del tempo di propagazione che determina ritardi diversi nella commutazione del segnale di uscita. La conseguenza dell'alea e' che il latch puo' immagazzinare un valore non corretto. I circuiti sincroni permettono di ovviare al problema.

22 Circuiti latch asincroni

Elementi di memoria: circuiti che immagazzinano 1 bit, lo stato del circuito e' il contenuto della memoria, transizione di stato = cambiare stato. Abbiamo visto il circuito latch SR come esempio di un elemento di memoria asincrono: la transizione di stato e' solo dipendente dai dati in ingresso.

22.1 Latch SR realizzato con porta NOR

S	R	Q	$\neg Q$
1	0	1	0
0	0	1	0
0	1	0	1
0	0	0	1
1	1		

Table 24: Tabella di verita'

Il circuito si puo' trovare in 2 stati:

- Stato di SET: $Q=1$
- Stato di REST: $Q=0$

22.2 Latch SR con porte NAND

$\neg S$	$\neg R$	Q	$\neg Q$
0	1	1	0
1	1	1	0
1	0	0	1
1	1	0	1
0	0		

Table 25: Tabella verita'

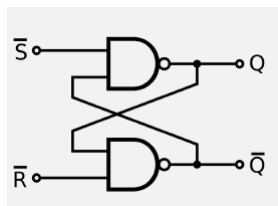


Figure 26: Latch NAND

23 Circuiti sincroni

Elementi di memoria sincroni: le transizioni di stato si hanno solo in corrispondenza di un segnale di sincronizzazione. I circuiti sincroni utilizzano un segnale di **clock**, generato da un apposito circuito che emette una serie di impulsi in modo periodico. Gli elementi di memoria possono effettuare una transizione di stato unicamente in corrispondenza di un impulso di clock.

23.1 Clock

Il segnale di clock è una sequenza di impulsi (onde quadre) con andamento periodico.

- Livello alto (1), livello basso (0)
- Impulsi di durata e distanza costante
- Ciclo di clock: l'intervallo di tempo che intercorre fra 2 impulsi consecutivi (periodo T)
- Frequenza del clock: numero di cicli di clock al secondo $f=1/T$. Si misura in Hertz.

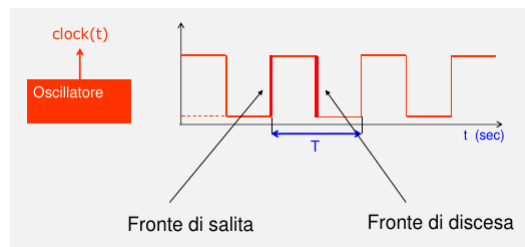


Figure 27: Clock

Esempio: Se il clock è a livello basso l'uscita è comunque 0. Se $D=0$ l'uscita sarà sempre 0. Se $D=1$, l'uscita sarà uguale al livello del clock.

23.2 Elemento di memoria sincrono: Latch D

Un elemento di memoria sincrono è il latch D. Vi è un unico ingresso per il dato (D). Il secondo ingresso è per il segnale di clock. La transizione di stato si ha solo quando il segnale del clock è alto: il valore binario all'ingresso D viene trasferito in uscita Q. Quando il livello del clock è basso, lo stato non cambia. È costituito da una latch SR, e da 2 porte AND abilitate dal segnale di clock. Esempi:

- $D=1$, $C=1$ allora $S=1$, $R=0$, $Q=1$ Stato di rest

- $D=0, C=1$ allora $S=0, R=1, Q=0$ Stato di rest
- $C=0$ allora $S=0, R=0$, lo stato non cambia

C	D	Stato futuro di Q
0	X	Nessun cambiamento
1	0	$Q=0$; stato di rest
1	1	$Q=1$; stato di set

Table 26: Latch D

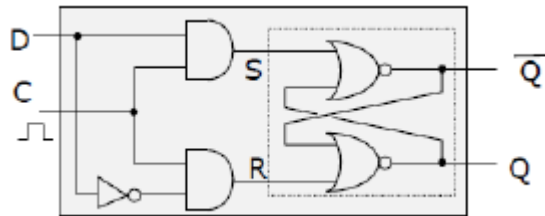


Figure 28: Latch D tramite latch SR

A differenza del latch SR non vi è più il problema della configurazione degli ingressi non consentita ($S=1, R=1$). Il latch D può essere realizzato anche con latch SR negati.

23.3 Problema della trasparenza

Quando il clock è a livello alto, il valore di D viene trasferito dall'ingresso all'uscita. Tuttavia il dato in ingresso può cambiare più volte. Non c'è modo per evitare questo, per cui il segnale non viene controllato e ci possono essere commutazioni indesiderate. Per questo si dice che il latch è **trasparente**: quando il clock è a livello alto il latch è come se non ci fosse.

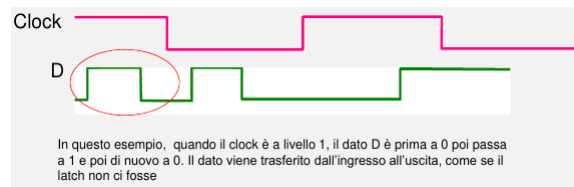


Figure 29: Problema della trasparenza

24 Flip-Flop

Il flip-flop permette di risolvere il problema della trasparenza. Esistono diversi tipi di flip-flop. Ci limitiamo a considerare un solo tipo. Flip-flop **attivi sui fronti del clock**: si ha una transizione di stato solo in corrispondenza delle transizioni del segnale di clock, quando il clock transita da 0 a 1 (attivo sul fronte di salita) oppure da 1 a 0 (attivo sul fronte di discesa).

24.1 Flip-flop tipo D

Attivo sul fronte di discesa e' costituito da 2 latch D in cascata e un invertitore disposti come riportato. Ha una configurazione "master-slave": il latch sulla sinistra e' il master, quello a destra lo slave. L'uscita del master e' l'ingresso dello slave $Q_m = D_s$. L'invertitore serve ad invertire il livello del clock, per cui se e' alto abilita il master ma non lo slave e viceversa.

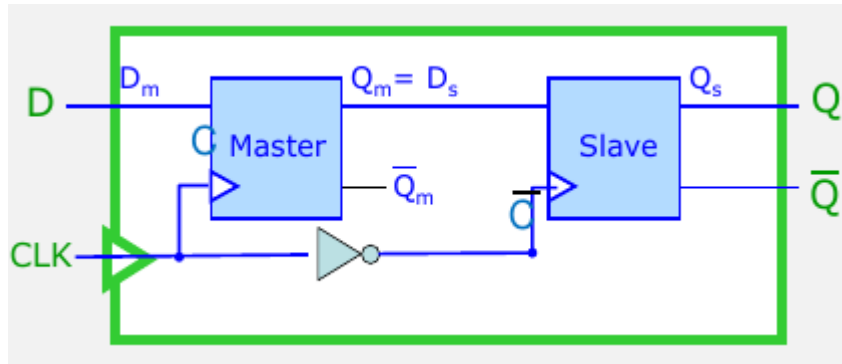


Figure 30: Flip-Flop di tipo D

- Clock alto: il master e' trasparente, quindi il valore d'ingresso D_m viene trasferito all'uscita Q_m . Il valore di Q_m rimane tuttavia bloccato per tutto il tempo in cui il clock e' alto.
- Clock basso: lo slave diventa trasparente quindi il valore di Q_m in ingresso e' trasferito all'uscita Q .

25 Registri

I flip-flop sono usati per immagazzinare 1 bit. Un registro e' costruito da un insieme di flip-flop, ed eventualmente anche da porte logiche/rete combinatoria. Un registro a n bit e' costituito da n flip flop che memorizzano n bit. Ad esempio un registro a 32 bit permette di memorizzare dati rappresentati con 32 bit. Le operazioni sui registri (in genere su ogni elemento di memoria) sono di lettura

di un registro (la determinazione dello stato del registro) e la scrittura di un registro (memorizzare un dato nel registro).

25.1 Un semplice registro

Registro a 3 bit. I circuiti F1, F2, F3 sono flip-flop di tipo D; i segnali D1, D2, D3 sono gli ingressi D dei flip flop. Ad ogni ciclo di clock viene memorizzato il valore degli ingressi D.

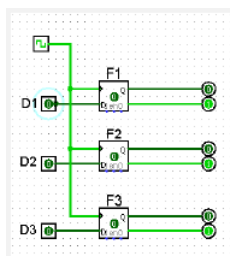


Figure 31: Registro semplice

In questo caso i bit in ingresso vengono memorizzati durante il ciclo di clock solo se il segnale ENABLE e' 1. In questo modo si puo' controllare quando effettuare la scrittura in un registro.

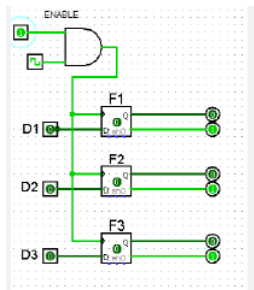


Figure 32: Registro con segnale di abilitazione

26 Introduzione

Componente centrale di un computer e' il processore. Il processore esegue istruzioni espresse in un linguaggio binario. L'insieme delle istruzioni che il processore e' in grado di eseguire costituisce l'architettura dell'insieme di istruzioni (ISA). L'ISA fornisce una interfaccia tra il sw e l'hw, nascondendo i dettagli implementativi dell'hw (circuiti). Per rendere piu' facilmente utilizzabili le istruzioni dell'ISA, si usa un linguaggio simbolico, detto linguaggio **assembly**.

27 Linguaggio macchina

Linguaggio ad alto livello: le istruzioni sono espresse in modo indipendente dal processore. Il programma costituisce il codice sorgente. Linguaggio macchina: le istruzioni sono espresse in binario e sono direttamente eseguite dal processore. Un codice sorgente per essere eseguito deve essere trasformato in linguaggio macchina.

28 Modello generale di computer

Il modello classico della struttura hw e del funzionamento di un calcolatore viene anche denominato architettura di Von Neumann. Caratteristiche fondamentali:

- Componenti:
 - Processore (Central Processing Unit - CPU): esegue le istruzioni specificate nei programmi. I programmi sono costituiti da istruzioni in formato binario.
 - Memoria Centrale: contiene programmi e dati in formato binario
 - Dispositivi di ingresso/uscita (I/O): dispositivi tramite i quali avviene la comunicazione con l'esterno.
- Un programma per essere eseguito si deve trovare in memoria centrale. La memoria contiene sia dati che istruzioni. La memoria e' leggibile e scrivibile.
- Le istruzioni di un programma sono eseguite in modo sequenziale, una di seguito all'altra.

Il ciclo fetch-and-execute del processore: le istruzioni in linguaggio macchina del programma in esecuzione risiedono in memoria centrale e sono disposte in sequenza nell'ordine in cui sono scritte dal programmatore. Il processore preleva un'istruzione alla volta e la esegue interagendo se richiesto con i dispositivi di I/O :

- Fetch: preleva la istruzione dalla memoria centrale
- Execute: decodifica la istruzione ed esegui

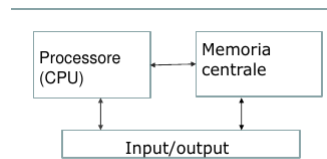


Figure 33: Modello generale di computer

- Ripeti il ciclo

Le istruzioni sono elaborate in sequenza, una di seguito all'altra

28.1 Processore (CPU)

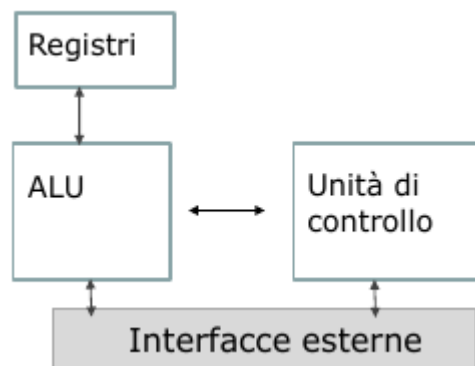


Figure 34: CPU

- Unità di controllo: 'cuore' del processore e' il componente che controlla la esecuzione delle istruzioni (rete combinatoria)
- ALU (unità aritmetico-logica): esegue le operazioni aritmetiche e logiche richieste dalla unità di controllo
- Registri: la memoria locale del processore
- Interfacce esterne: unità che gestiscono la comunicazione con memoria centrale e I/O

28.2 La memoria centrale

La memoria centrale e' un insieme di locazioni (anche parole o celle) di memoria. Le locazioni contengono l'informazione binaria che rappresentai i dati e le

istruzioni. Ogni locazione e' costituita da un numero prefissato di bit. Tale numero definisce la dimensione della locazione. Ogni locazione di memoria e' identificata da un intero (senza segno), l'indirizzo di memoria. La memoria possiamo vedere come un array costituito da una sequenza di locazioni. L'accesso ad una locazione (per operazioni di lettura e scrittura) avviene specificando l'indirizzo.

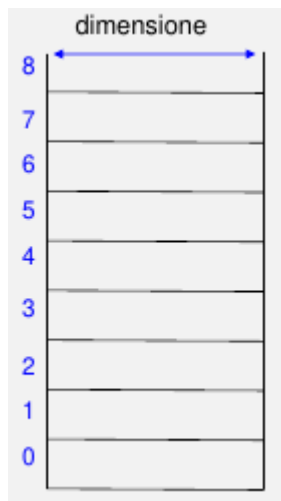


Figure 35: Locazioni di memoria con relativo indirizzo

28.3 Aspetti tecnologici: memorie

L'accesso alla memoria centrale avviene in un tempo prefissato che non dipende dalla posizione della locazione (tempo di accesso alla memoria). Memorie ad accesso casuale (RAM): si tratta di una memoria volatile, il contenuto viene perso se manca l'alimentazione, si accede a tutte le celle con lo stesso tempo. Memorie a sola lettura (ROM): trattiene i dati per un tempo indefinito; la memoria e' non volatile. Memorie RAM :

- Ram dinamica (DRAM): memorizza un bit come carica di un condensatore. I bit devono essere ricaricati (memoria rinfrescata)
- RAM statica (SRAM): memoria costruita con circuiti, piu' veloce della DRAM. I bit non hanno bisogno di essere riscritti.

Memoria di massa: memoria non volatile utilizzata per conservare programmi e dati. Ad esempio dischi magnetici, memorie flash. Le memoria di massa sono dispositivi di I/O.

28.4 L'architettura dell'insieme delle istruzioni (ISA)

Instruction Set Architecture (ISA) - architettura in breve- comprende tutte le informazioni che consentono di scrivere un programma in linguaggio macchina:

- Le istruzioni che il processore può eseguire e il loro formato binario
- i registri contenuti nel processore
- caratteristiche della memoria centrale

La architettura descrive le caratteristiche del processore in modo indipendente dalla attuale implementazione, cioè non dice quali circuiti ci sono nel processore, ma ci dice come programmare il processore a patto che i registri siano quelli. L'implementazione è l'hw che realizza la descrizione astratta dell'architettura. Quindi vi possono essere processori diversi (di diversi fornitori) che si basano sulla stessa ISA ma forniscono implementazioni differenti. Architettura MIPS, X86, ARM.

28.5 Classi di architetture

Fino a metà degli anni 80 il tipo di architettura dominante è quella CISC. L'idea era quella di mettere a disposizione più istruzioni possibili per rendere il processore più potente possibile, il problema è che avendo molte istruzioni (anche molto diverse una dall'altra) hanno come effetto il fatto che il processore diventa più complesso e questa complessità poi si ripercuote sull'efficienza. La maggior parte del tempo di elaborazione viene impiegato per eseguire poche istruzioni semplici, l'idea delle architetture RISC è di ottimizzare le istruzioni semplici. Si ottiene un set di istruzioni ridotto ma molto efficienti. RISC: operazioni complesse scomposte in serie di istruzioni semplici, numero molto ridotto di formati per le istruzioni, CPU semplice, si riducono i tempi di esecuzione delle singole istruzioni, il risultato sono poche istruzioni ma veloci.

29 Architettura MIPS

Si tratta di un'architettura RISC. Il primo processore MIPS commerciale (R2000).

29.1 Memoria centrale

La dimensione delle celle di memoria è 8 bit (1 byte), ogni byte ha un indirizzo di memoria rappresentato con 32 bit. Quindi la dimensione massima della memoria è data dal numero massimo di byte indirizzabili: 2^{32} bytes (4 Giga Byte). I byte sono raggruppati in parole di 4 byte (words). Le parole devono essere allineate, ovvero devono iniziare ad un indirizzo che è multiplo di 4, ad esempio: 0, 4, 8... . In termini astratti vediamo la memoria come un array di byte in cui è possibile prendere anche gruppi di 4.

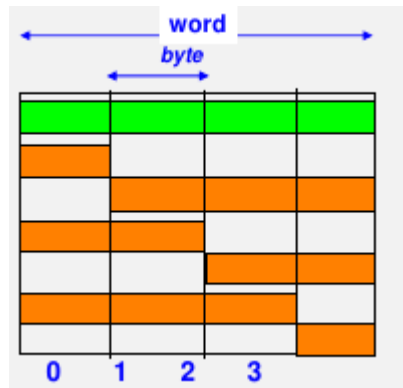


Figure 36: MIPS

29.1.1 Disposizione dei byte in una parola

In generale, i byte all'interno di una parola possono essere disposti secondo due criteri diversi:

- Big-endian: byte più significativo con indirizzo più basso
- Little-endian: byte più significativo con indirizzo più alto.

Il MIPS usa big-endian.

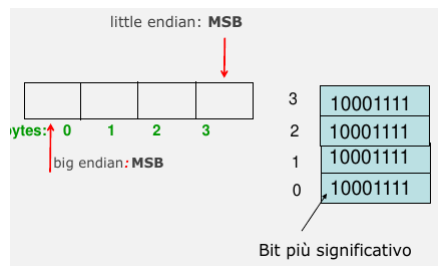


Figure 37: Disposizione dei byte in una parola

Esempio: per indicare l'indirizzo si usa normalmente la notazione esadecimale. Indirizzo di un byte: intero rappresentabile su 32 bit. ES: 0x 84B05801. Indirizzo di una parola: intero multiplo di 4. ES: 0x 84B05800. I byte della parola hanno indirizzo 0x 84B05800, 0x 84B05801, 0x 84B05802, 0x 84B05803. Il byte più significativo è quello a indirizzo più basso.

29.2 Processore

Banco di registri di uso generale: 32 registri da 32 bit. Sono Usati per memorizzare gli operandi e il risultato delle operazioni aritmetico-logiche effettuate

dalla ALU; memorizzare i dati trasferiti da/a memoria centrale. I registri sono numerati da 0 a 31 (indirizzo del registro). Le istruzioni possono leggere e scrivere nei registri specificati. La lettura non modifica il contenuto; la scrittura sovrascrive il contenuto. Il registro 0 e' un registro che contiene 0 e non puo' essere modificato. I registri sono l'unica risorsa che il programmatore puo' utilizzare.

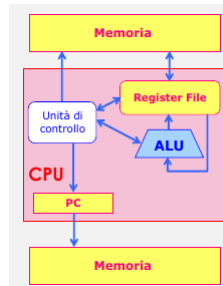


Figure 38: Architettura MIPS

29.2.1 Program Counter (PC)

Contiene l'indirizzo dell'istruzione corrente da aggiornare durante l'evoluzione del programma, in modo da prelevare dalla memoria la corretta sequenza di istruzioni. Le istruzioni sono parole (32 bit) memorizzate in sequenza a partire da indirizzi piu' bassi verso indirizzi piu' alti. Ciclo:

- Fetch: l'indirizzo dell'istruzione in PC viene utilizzato per prelevare l'istruzione corrente. Il valore di PC viene aggiornato all'istruzione successiva ($PC+4$)
- Execute: l'istruzione viene decodificata dall'unita' di controllo ed eseguita

29.3 Co-processor

In generale, i co-processor sono dispositivi in grado di effettuare operazioni specifiche, su richiesta della CPU, in modo molto efficiente. Ad esempio, il coprocessore matematico e' in grado di svolgere operazioni in virgola mobile. Un processore MIPS puo' utilizzare co-processor. Il coprocessore matematico comprende 32 registri di 32 bit (in precisione semplice) o 16 registri di 64 bit (in precisione doppia).

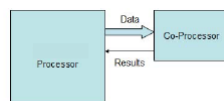


Figure 39: Coprocessori

30 Istruzioni linguaggio Assembly

Il linguaggio assembly e' specifico di ogni architettura. E' un linguaggio simbolico le cui istruzioni sono in stretta corrispondenza con le istruzioni in linguaggio macchina. Si tratta di un linguaggio piu' primitivo rispetto ad un linguaggio ad alto livello:

- Non ci sono costrutti per il controllo del flusso sofisticati (for, if, while)
- Istruzioni piu' restrittive (numero di operandi limitato)

L'assemblatore traduce un programma da assembly a linguaggio macchina.

30.1 Esempio

lw \$8, 32(\$19)	legge una parola di memoria nel registro 8
lw \$7, 36(\$19)	legge una parola di memoria nel registro 7
add \$6, \$8, \$7	scrive la somma del contenuto del reg 7 e reg 8 nel reg 6
sw \$6, 8(\$19)	scrive il contenuto del reg. 6 in una parola di memoria.//

Table 27: esempio

30.2 Traduzione dei programmi

Il compilatore trasforma il programma C (sintatticamente corretto) in un programma in un linguaggio assembly, un linguaggio simbolico le cui istruzioni sono in corrispondenza con le istruzioni in linguaggio macchina. L'assemblatore converte il programma assembly (sintatticamente corretto) in linguaggio macchina. Esistono linguaggi di programmazione interpretati o compilati, interpretati significa che un interprete fa la traduzione immediatamente ed esegue l'istruzione.

31 Introduzione

MIPS e' un esempio di architettura RISC. Riduzione e semplificazione delle istruzioni. Il linguaggio assembly MIPS (asm in breve) specifica in forma simbolica le istruzioni che sono direttamente eseguibili dal processore. In generale lo studio del linguaggio assembly permette di comprendere piu' a fondo il funzionamento di un processore.

32 Regole sintattiche di base

- Ogni riga del programma corrisponde ad una sola istruzione, L'istruzione specifica il nome simbolico dell'operazione (codice mnemonico, es: add) e gli operandi.
- La notazione \$N indica il registro N, dove N e' l'indirizzo compreso fra 0 e 31, il registro 0 contiene 0 e non puo' essere modificato
- Le costanti si indicano in decimale o in esadecimale
- questo simbolo indica un commento

33 Classi di istruzioni MIPS

L'architettura delle istruzioni MIPS comprende istruzioni:

- Aritmetiche
- Logiche e di scorrimento
- Trasferimento dati (da/a memoria centrale)
- Salti (controllo del flusso di istruzioni)

Tutte le istruzioni sono rappresentate su 32 bit. Questa regolarita' e' in linea con la filosofia RISC.

33.1 Istruzioni aritmetiche

add, sub, addi (add immediate). Operandi in 3 registri. Le istruzioni che terminano con la i possono usare le costanti.

Sintassi	Significato
add rd, rs1, rs2	scrivi nel registro rd la somma rs1+rs2
	Scrivi nel registro rd la differenza rs1-rs2
addi rd, rs1, costante	Scrivi nel registro rd la somma rs1+costante

Table 28: Caption

- rd: registro destinazione
- rs: registro sorgente

Le costanti sono rappresentate con 16 bit. A questo livello di astrazione il programmatore non sa che circuiti vengono usati per fare le operazioni. Per copiare un il contenuto di un registro in un altro si usa l'operazione add. Un'unica istruzione viene usata per scopi di versi. E' la filosofia RISC di ridurre il numero di istruzioni.

33.1.1 Esempio1

Considerando le istruzioni:

```
add $4, $8, $2
add $4, $4, $4
sub $4, $4, $8
addi $4, $4, 10
```

Si assuma che i registri contengano i seguenti dati:

- \$4=5
- \$8=3
- \$2=6

Determinare il contenuto del registro 4 al termine dell'esecuzione delle istruzioni.

33.1.2 Esempio2

Per copiare il contenuto di un registro in un altro registro si usa l'istruzione add. Esempio:

```
add $4, $8, $0  #$4 in $8
add $4, $0, $0  #$4 in 0
```

33.2 Tipi di dato

Le operazioni aritmetiche sono effettuate fra interi. Gli interi possono essere con segno e senza segno. Vi sono istruzioni distinte per operare con i due tipi di dato. Le seguenti istruzioni interpretano il contenuto dei registri come interi rappresentati in complemento a 2: **add**, **sub**, **addi**, intervallo di rappresentazione: $[-2^{31}, 2^{31} - 1]$. Al contrario **addu**, **subu**, **addiu** (u sta per unsigned) operano su interi senza segno, intervallo di rappresentazione: $[0, 2^{32} - 1]$. 0xFFFFFFFF e' il max numero rappresentabile. L'istruzione subi non esiste. Per sottrarre un numero N in complemento a 2 si somma in numero opposto -N. add \$4, \$4, -10.

33.3 Overflow

Lo possono generare solo add, addi e sub. L'overflow genera un'eccezione. Un'eccezione e' un evento che determina l'interruzione del programma in esecuzione e il trasferimento del controllo ad un altro programma, il gestore delle eccezioni.

34 Corrispondenza con linguaggio ad alto livello

Si consideri il seguente assegnamento (in linguaggio ad alto livello): $A = B + C + D + 4$. Si assuma che le variabili siano associate ai registri come segue:

- A: \$7
- B: \$3
- C: \$4
- D: \$5

Soluzione in assembly:

```
add $7, $3, $4
add $7, $7, $5
addi $7, $7, 4
```

35 Istruzioni di scorrimento e logiche

Si tratta di istruzioni che permettono di operare su singoli bit di una parola:

- Istruzioni di scorrimento: sll, srl
- Istruzioni logiche: and, andi, or, ori, nor, xor

35.1 Istruzioni di scorrimento SLL/SRL

sll (shift left logical), srl (shift right logical). Le istruzioni di scorrimento o shift, permettono di spostare a destra o sinistra n bit di una parola riempiendo le posizioni vuote con 0.

Sintassi	Esempio
sll rd, rs, costante	sll \$4, \$5, 2
srl rd, rs, costante	srl \$4, \$5, 2

Table 29:

La costante indica il numero di cifre da spostare. Scorrimento di i cifre:

- a sinistra equivale alla moltiplicazione per 2^i
- a destra equivale alla divisione intera per 2^1 (restituisce la parte intera)

35.2 Esempio

Si assuma che `\$3` contenga la sequenza binaria:

```
00000000000000000000000000000011
```

```
sll $2, $3, 4
```

L'istruzione `sll` effettua uno shift a sinistra di 4 posizioni del contenuto di `$3`

```
0000000000000000000000000110000
```

```
srl $2, $3, 1
```

Al termine dell'istruzione, il registro `$2` contiene:

```
00000000000000000000000000000001
```

35.3 Istruzioni logiche AND/ANDI

Confronta 2 parole bit per bit: se entrambi i bit sono a 1 allora il bit del risultato e' 1 altrimenti 0.

Sintassi	Esempio	Significato
<code>and rd, rs1, rs2</code>	<code>and \$4, \$2, \$3</code>	metto rs1 and rs2 in rd
<code>andi rd, rs1, costante</code>	<code>and \$4, \$2, 4</code>	metto rs1 and costante _{es} in rd

Table 30:

35.3.1 Esempio

```
andi $4, $2, 0xfffe
```

```
$2: 0000 0000 0000 0000 1000 1001 0011 1011
```

```
Costante binaria: 1111 1111 1111 1110
```

```
$4 — 0000 0000 0000 0000 1000 1001 0011 1010
```

L'istruzione permette di porre a 0 i bit selezionati tramite l'uso di una maschera in cui sono a 1 solo i bit di interesse.

35.4 Istruzioni logiche OR/ORI

Confronta 2 parole bit per bit: se almeno uno dei due bit e' a 1 allora il bit risultato e' 1 altrimenti 0.

Sintassi	Esempio	Significato
<code>or rd, rs1, rs2</code>	<code>or \$4, \$2, \$3</code>	metto rs1 and rs2 in rd
<code>ori rd, rs1, costante</code>	<code>ori \$4, \$2, 4</code>	metto rs1 and costante _{es} in rd

Table 31:

35.4.1 Esempio

```
ori $4, $2, 1
$2: 0000 0000 1001 1011 0000 0000 1001 0010
Costante: 0000 0000 0000 0001
$4 — 0000 0000 1001 1011 0000 0000 1001 0011
```

L'istruzione permette di porre a 1 i bit selezionati tramite l'uso di una maschera in cui sono a 1 solo i bit di interesse.

36 Istruzioni di trasferimento dati

IL processore contiene un numero limitato di registri generali per memorizzare i dati, mentre un programma puo' avere la necessita' di utilizzare molti dati (ad esempio variabili). E' pertanto necessario poter disporre di una memoria piu' ampia. Per questo viene utilizzata la memoria centrale. Le variabili presenti nei programmi ad alto livello vengono mappate locazioni della memoria centrale (non nei registri). Quindi la memoria non contiene solo istruzioni ma anche i dati. Per identificare tale locazioni, non si usa un nome, ma un indirizzo. Le istruzioni di trasferimento dati trasferiscono dati fra la memoria centrale e i registri della CPU.

36.1 Memoria MIPS

La memoria centrale e' indirizzata al byte. Questo significa che esiste un indirizzo distinto per ogni byte. L'indirizzo e' un numero espresso su 32 bit. Lo spazio , degli indirizzi e' dato dall'intervallo di rappresentazione, quindi la dimensione massima e' 4gb. Operazione di lettura e di scrittura su byte o parola. Le parole (32 bit) devono avere un indirizzo multiplo di 4 (vincolo di allineamento).

37 Ciclo fetch -execute

La memoria centrale contiene il programma in esecuzione. Le istruzioni sono rappresentate da word (32 bit) e sono disposte in sequenza da indirizzi piu' bassi a indirizzi piu' alti, rispettando il vincolo di allineamento. Durante il ciclo fetch-execute, il processore:

- Legge la prossima istruzione da eseguire dalla memoria centrale
- Interpreta la istruzione e la esegue
- Ripete il ciclo

Il **Program Counter** (PC) e' il registro che contiene l'indirizzo della istruzione da eseguire (istruzione corrente)

37.1 Program Counter

L'indirizzo della prossima istruzione da eseguire può essere determinato in modo diverso:

- Il contenuto del PC viene incrementato automaticamente di 4 per eseguire la prossima istruzione in sequenza : PC diventa PC+4
- Il contenuto del PC viene modificato a seguito di un'istruzione di salto, quindi da programma.

38 Istruzioni in sequenza

Frammento del programma: Nell'esempio, la prima istruzione ha indirizzo 0x23456000.

Indirizzo istruzione	Istruzione ASM
0x23456000	add \$6, \$5, \$0
0x23456004	add \$7, \$4, \$0
0x23456008	sub \$7, \$7, \$6

Table 32: Esempio

L'indirizzo viene aggiornato all'istruzione successiva sommando 4.

39 Istruzioni di salto

Classe delle istruzioni che consentono di alterare l'ordine sequenziale con cui sono eseguite le istruzioni. Si tratta di istruzioni che modificano il contenuto del registro PC. Nei linguaggi ad alto livello il flusso delle istruzioni può essere modificato utilizzando i costrutti di controllo quali i costrutti iterativi e condizionali. Questi costrutti vengono realizzati in assembly tramite istruzioni di salto. La classe delle istruzioni di salto comprende due categorie:

- **Istruzioni di salto condizionato:** la modifica del PC e quindi il salto si verifica solo se la condizione logica specificata nell'istruzione è vera.
Istruzioni di salto incondizionato: il PC viene comunque modificato e il salto viene effettuato all'indirizzo specificato

39.1 Istruzioni di salto condizionato: beq

Istruzione **beq**: branch-on-equal.

Sintassi	Esempio
beq r1, r2, L1	beq \$3, \$4, etich

Table 33:

L'istruzione confronta i valore dei registri r1 e r2 e determina se questi valori sono uguali. Se i valori sono uguali viene effettuato un salto ala istruzione etichettata L1, altrimenti la prossima istruzione da eseguire e' quella in sequenza. L'etichetta (label) e' un nome simbolico usato per identificare una istruzione assembly. L'assemblatore converte le etichette in valori numerici poi usati per determinare il nuovo valore del PC.

39.1.1 Etichette

L'associazione ad una istruzione e' effettuata usando la seguente notazione: **etichetta: istruzione**. La etichetta puo' essere utilizzata all'interno di una istruzione di salto. Il salto puo' essere effettuato in avanti, verso istruzioni con indirizzo piu' alto, o verso istruzioni con indirizzo piu' basso.

39.2 Istruzioni di salto condizionato: bne

Istruzione **bne**: branch-on-not-equal.

Sintassi	Esempio
bne r1, r2, L1	bne \$3, \$4, etich

Table 34:

L'istruzione confronta i valori dei registri r1 e r2 e determina se questi valori sono diversi. Se i valori sono diversi viene effettuato un salto alla istruzione etichettata L1, altrimenti la prossima istruzione da eseguire e' quella in sequenza.

39.2.1 Esempio

Considerare il frammento di programma:

```
lw $2, 0($8)
lw $3, 4($8)
bne $2, $3, etich
addi $3, $0, 1000
etich: addi $3, $3, -10
```

Si richiede di determinare il valore di \$3. Soluzione:

```
lw $2, 0($8) # nel registro $2 c'e' 4
lw $3, 4($8) # nel registro 3 c'e' 10
bne $2, $3, etich # se il contenuto di 2 e' diverso da 3 salta a etich
addi $3, $0, 1000
etich: addi $3, $3, -10 # in 3 c'e' 0
```

39.3 Istruzione di confronto SLT

Le istruzioni beq, bne permettono di verificare se due valori sono uguali o meno ed effettuare il salto di conseguenza. La istruzione SLT confronta due numeri e stabilisce se uno è minore dell'altro: slt (set on less than).

Sintassi	Esempio
slt rd, rs1, rs2	slt \$3, \$4, \$5

Table 35:

Se il numero nel primo registro sorgente (rs1) è minore del numero nel secondo registro sorgente (rs2), allora il registro destinazione (rd) prende valore 1 altrimenti 0. I numeri che vengono confrontati sono interi con segno.

39.3.1 Esempio

Considerare il frammento di programma:

```
slt $4, $2, $3  # if $2<$3 then in $4 mettiamo 1 else mettiamo 0
bne $4, $0, minore
addi $5, $0, -1000
minore: addi $5, $0, 1000
```

39.4 Istruzioni di salto incondizionato J

Se incondizionato, il salto ad una diversa istruzione NON è subordinato al verificarsi di una condizione logica. Considerando l'istruzione J (jump-to).

Sintassi	Esempio
j etich	j exit

Table 36:

L'istruzione effettua un salto alla istruzione etichettata etich.

40 Rappresentazione binaria delle istruzioni

Le istruzioni di un programma sono caricate in memoria centrale come sequenze binarie- **istruzioni macchina** - espresse in linguaggio macchina. Il programma binario costituisce il **codice macchina**. Si definisce:

- Campo dell'istruzione: una parte dell'istruzione
- Formato (binario) dell'istruzione: la scomposizione in campi di un'istruzione

L'architettura delle istruzioni (ISA) specifica non solo le istruzioni ma anche il formato delle stesse. Consideriamo il formato delle istruzioni MIPS e descriviamo il formato di alcune delle istruzioni viste fino ad ora.

41 Formato delle istruzioni MIPS

Le istruzioni hanno lunghezza 32 bit. Sono definiti 3 formati:

- Formato R (registro): istruzioni aritmetiche, logiche, scorrimento
- Formato I (immediato): aritmetiche e logiche con operandi immediati, trasferimento dati, salto condizionato
- Formato J (jump): istruzioni di salto incondizionato

I formati sono riconosciuti tramite i 6 bit piu' significativi: campo **codice operativo**.

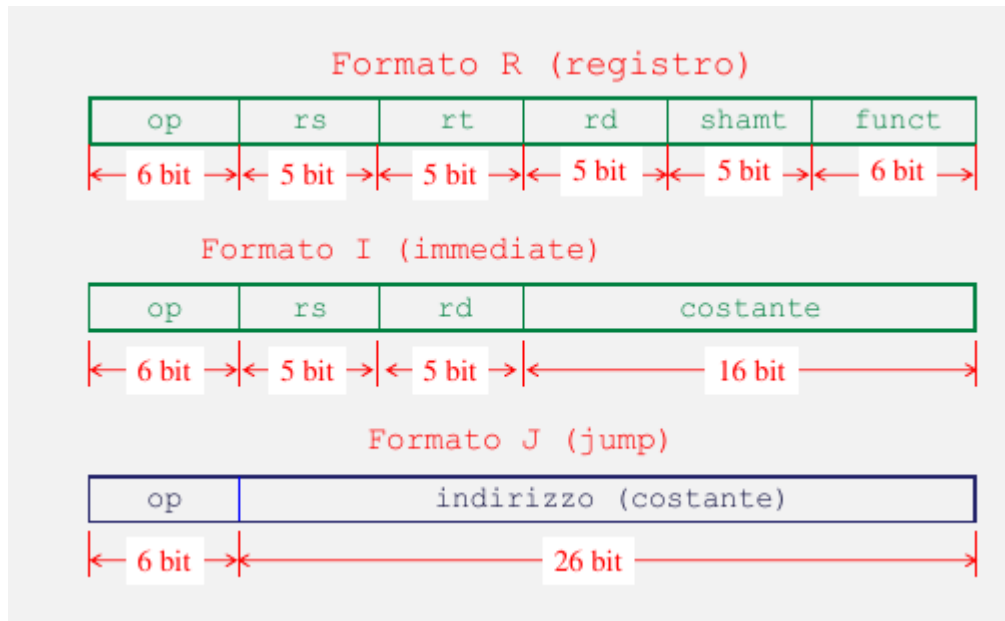


Figure 40: Formato istruzioni MIPS

41.1 Istruzioni in formato R

- add
- sub
- or/and/nor/xor
- srl/sll
- slt

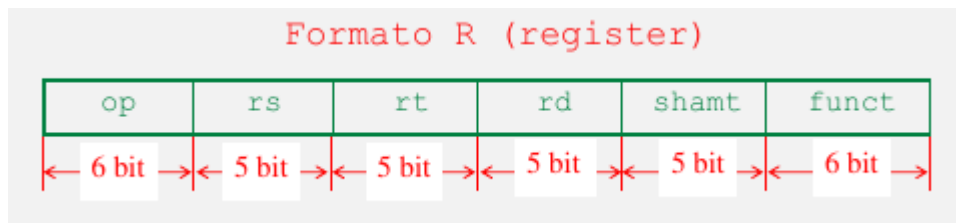


Figure 41: Formato R

- op : codice operatio = 0 (6 bit); questo vale per tutte le istruzioni di questa classe
- rs : registro contenente il primo operando sorgente (5 bit)
- rt: registro contenente il secondo operando sorgente (5 bit)
- rd: registro destinazione contenente il risultato (5 bit)
- shamt: shift amount (usato nella istruzione di scorrimento) (5 bit)
- funct: campo funzione indica la variante specifica dell'operazione (6 bit)

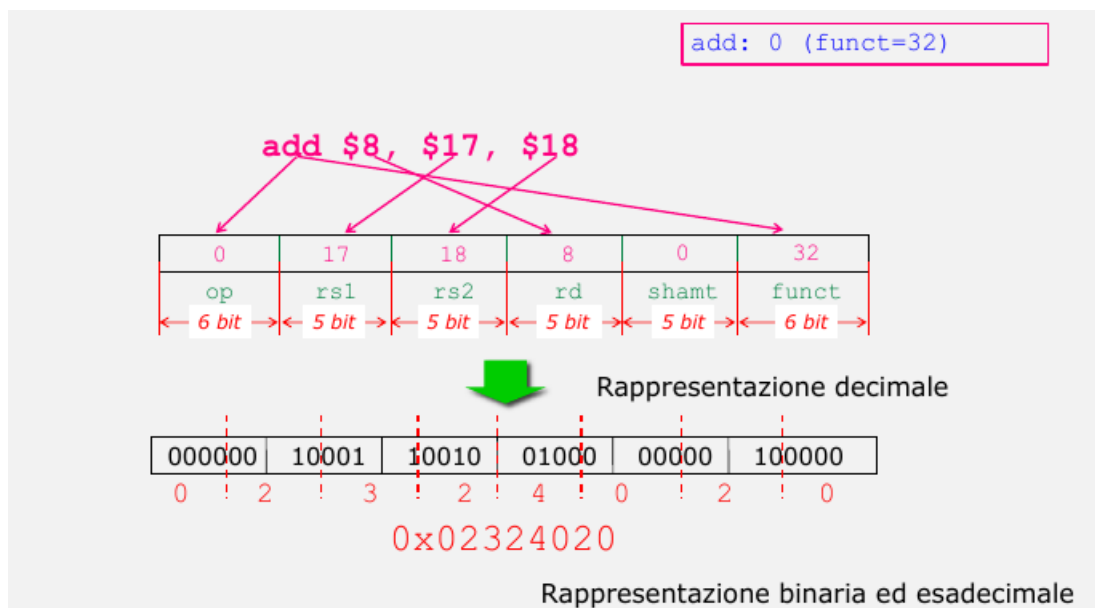


Figure 42: Istruzione ADD

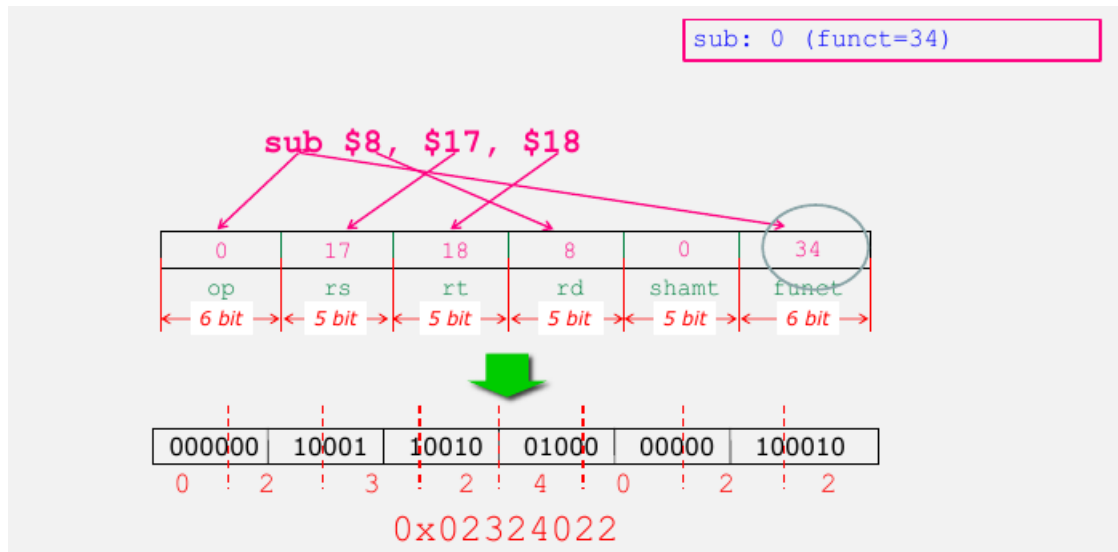


Figure 43: Istruzione SUB

41.1.1 Esercizio

Esprimere in forma simbolica la seguente istruzione espressa in forma binaria: 00000000001000001111100000100000. Si procede come segue: si considera il codice operativo, Se e' 0 allora la istruzione ha formato R. Si individuano quindi i campi e si interpretano: 000000|00001|00000|11111|00000|100000 : add \$3, \$1, \$0. add: op=0, funct=32.

41.1.2 Istruzioni di scorrimento

Hanno formato R. Tuttavia uno degli operandi e' una costante rappresentata nel campo **shamt**.

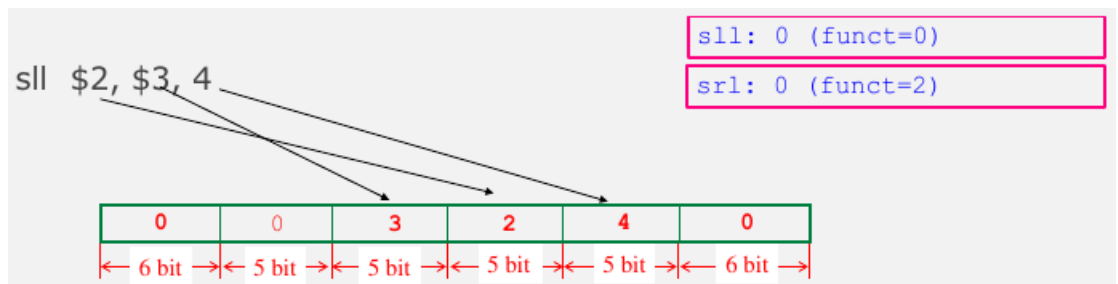


Figure 44: Esempio

41.2 Formato I

Istruzioni che contengono un operando immediato:

- Trasferimento dati: lw, sw
- Aritmetico logiche con operando immediato: addi, andi, ori
- Salto condizionato: beq, bne

41.2.1 Trasferimento dati

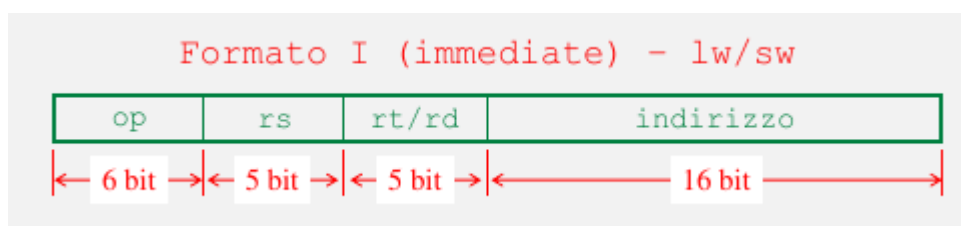


Figure 45: Trasferimento dati

- op: codice operativo
- rs: registro BASE
- rt/rd: indica il secondo registro (sorgente (sw) o destinazione (lw))
- Indirizzo: riporta lo spiazzamento (offset)

16 bit di spiazzamento. Intervallo: lo spiazzamento ricade nell'intervallo $[-2^{15}, +2^{15} - 1]$

41.2.2 Esercizio

Interpretare le seguenti istruzioni espresse in forma binaria:

- 10001101000000010000000000000000
- 00000000001000001111100000100000

Si interpretano a partire dal codice operativo:

- 100011|01000|00001|0000000000000000 = lw \$1, 0(\$8)
- 000000|00001|00000|11111|00000|100000 = add \$1, \$31, \$0

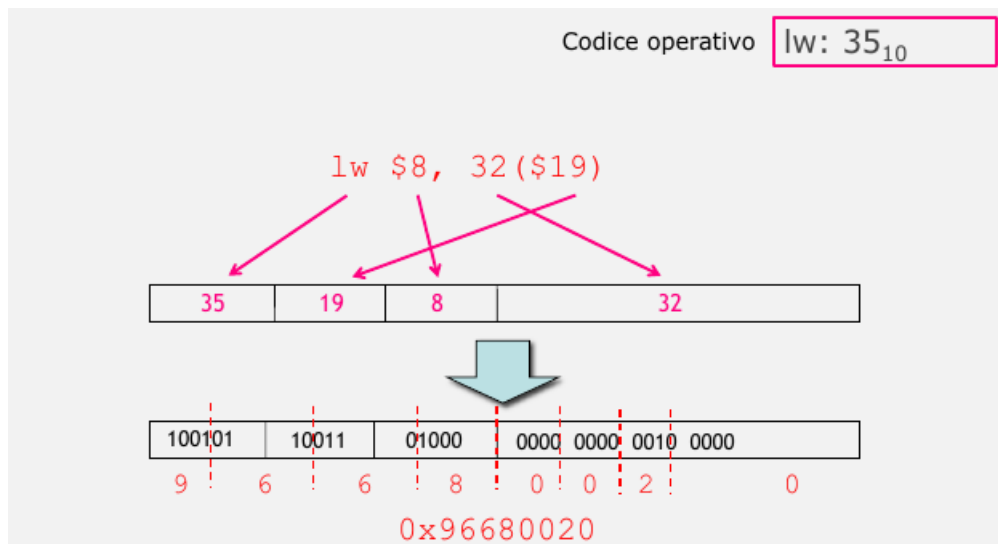


Figure 46: Istruzione lw

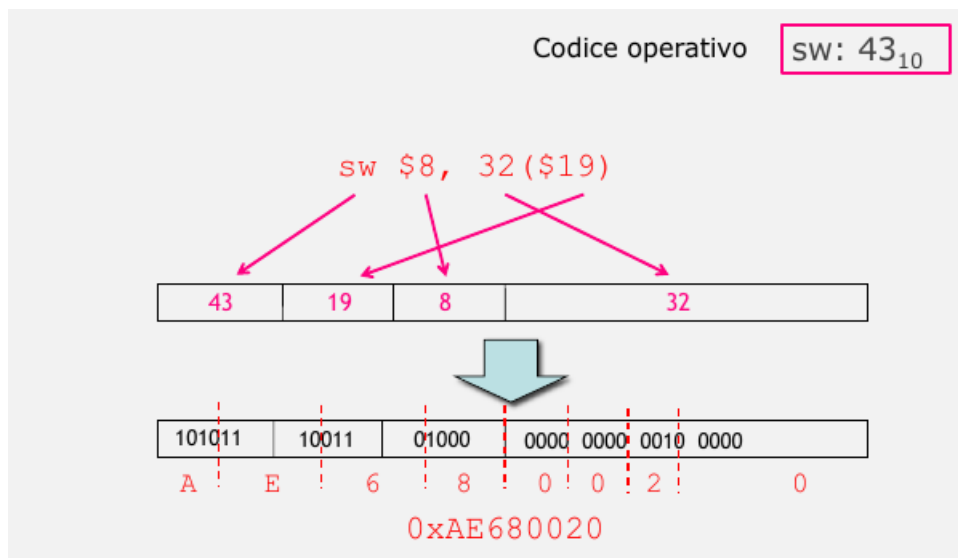


Figure 47: Istruzione sw

41.2.3 Istruzioni aritmetiche e logiche con op. immediato

- op: identifica il tipo di istruzione
- rs: indica il registro sorgente (primo operando)



Figure 48: Operatore Immediato

- rd: indica il registro destinazione
- costante riporta il secondo operando (costante)

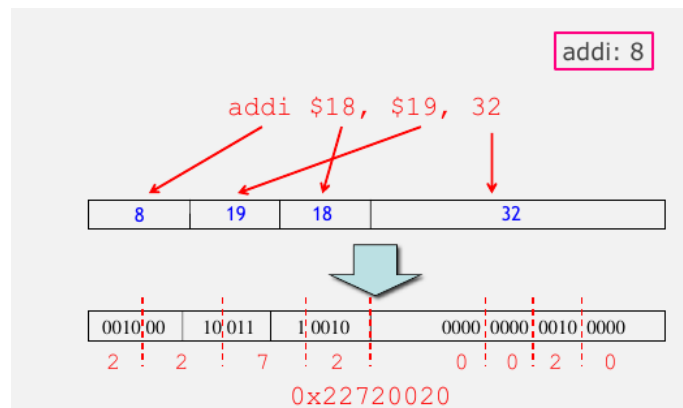


Figure 49: Istruzione addi

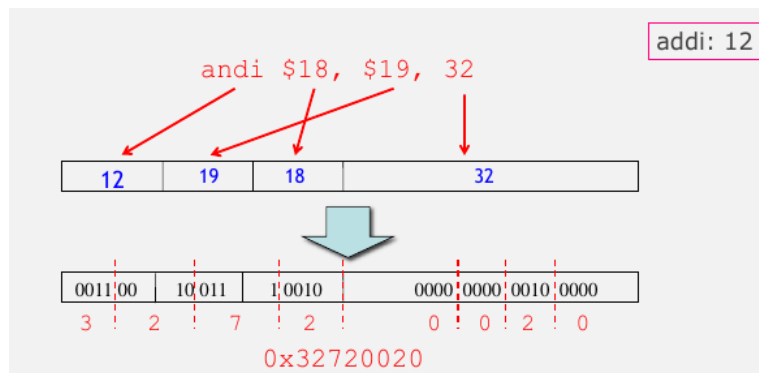


Figure 50: Istruzione andi

41.2.4 Nota sulle costanti

La costante specificata in addi viene estesa dall'istruzione a 32 bit replicando il bit di segno. La costante specificata in addi/ori viene estesa dall'istruzione a 32 bit con i 16 bit più significativi a 0.

41.2.5 Istruzioni di salto condizionato

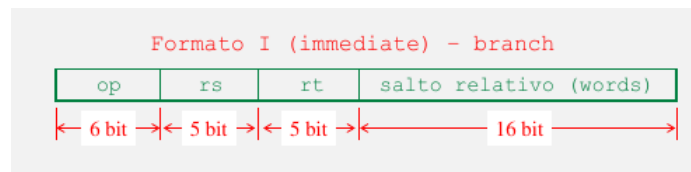


Figure 51: Salto condizionato

- op codice operativo
- rs indica il primo registro da confrontare
- rt indica il secondo registro da confrontare
- salto_relativo riporta l'ampiezza del salto rispetto al PC
 - L'ampiezza del salto è espressa in parole (words)
 - La ampiezza è calcolata rispetto all'istruzione successiva a quella corrente, ossia PC+4 (e non dall'indirizzo dell'istruzione corrente)

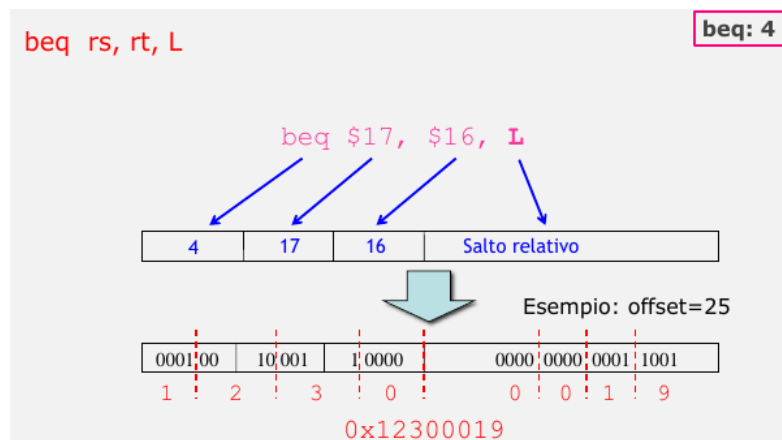


Figure 52: Istruzione beq

Il salto relativo al PC e' rappresentato in complemento a due. La distanza varia tra: -2^{15} e $+2^{15} - 1$ parole. Una distanza negativa indica un salto all'indietro. L'indirizzo al byte corrisponde all'indirizzo di parola moltiplicato per 4 (si aggiungono due bit 00 come bit meno significativi per ottenere l'indirizzo al byte): intervallo degli indirizzi: $[-2^{17} / +2^{17} - 1]$ bytes.

41.3 Formato J

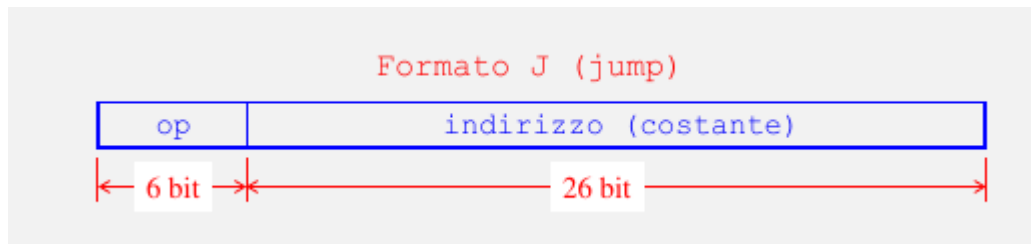


Figure 53: Formato J

Istruzioni di salto condizionato:

- op: codice operativo
- indirizzo (26 bit): riporta una parte dell'indirizzo assoluto di destinazione del salto

I 26 bit del campo indirizzo rappresentano parole. Per ottenere l'indirizzo al byte si moltiplica per 4. Un indirizzo e' tuttavia rappresentato su 32 bit. Gli altri 4 bit sono piu' significativi del PC (il PC viene modificato solo per 28 bit meno significativi)

41.3.1 Costruzione dell'indirizzo

Il campo indirizzo di 26 bit rappresenta un indirizzo di parola. L'indirizzo del byte si ottiene: $[\text{indirizzo di parola} \times 4] = [\text{indirizzo di parola} \mid 00]$ e aggiungendo i 4 bit piu' significativi del PC

42 Riepilogo

3 formati diversi: R, I, J. Il formato e' riconosciuto dalla CPU tramite il codice operativo(op)/funzione (funct).

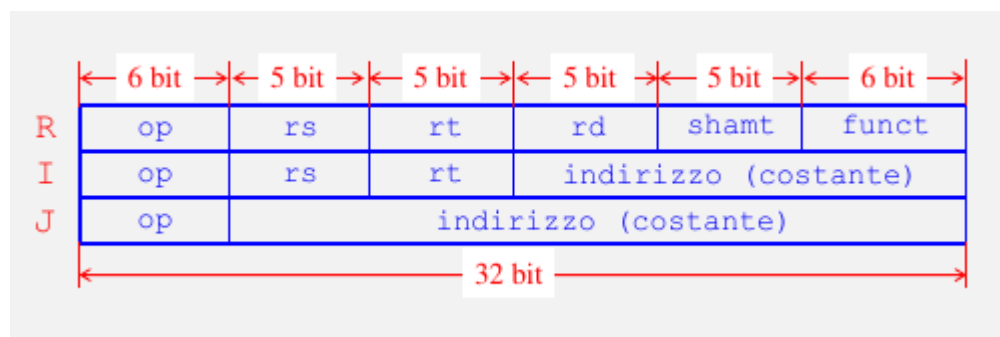


Figure 54: Formati

43 Introduzione

La architettura delle istruzioni di un processore non specifica la implementazione hardware. E' dunque possibile avere implementazione diverse per la stessa architettura. Vogliamo discutere una possibile implementazione hardware di un processore semplificato in grado di eseguire un sottoinsieme delle istruzioni MIPS:

- Istruzioni logico-matematiche (add, sub, and, or, sit)
- Accesso alla memoria (lw) e scrittura (sw)
- Istruzioni di salto condizionato (branch) e incondizionato (J)

44 CPU: unita' di controllo e unita' di elaborazione

Da un punto di vista hardware, la CPU e' costituita da due unita': l'unita' di controllo (control unit) e la unita' di elaborazione (o data path). L'unita' di controllo riceve in input un'istruzione, la interpreta e genera segnali di controllo per istruire l'unita' di elaborazione. I segnali di controllo sono diversi dai segnali che portano i dati. La unita' di elaborazione esegue le istruzioni utilizzando elementi di stato ed elementi combinatori tra loro connessi.

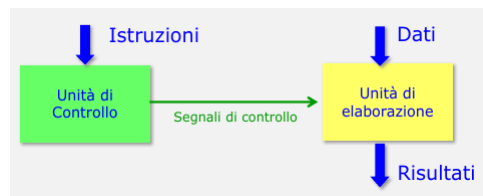


Figure 55: Connessione unita' di controllo e elaborazione

45 Unita' di elaborazione

Elementi di stato: PC, registri generati. Costituiti da elementi di memoria.
Elementi combinatori:

- ALU (unita' aritmetico logica): esegue le operazioni aritmetiche e logiche. Tutte le istruzioni, fatta eccezione per quella di Jump, quindi anche quelle di trasferimento da/a memoria e salto condizionato, richiedono l'utilizzo della ALU.
- Sommatore (add): calcolo di somme
- Memoria istruzioni: lettura di una istruzione dalla memoria centrale

- Memoria dati: scrittura/lettura di un dato in memoria centrale sulla base dell'indirizzo.

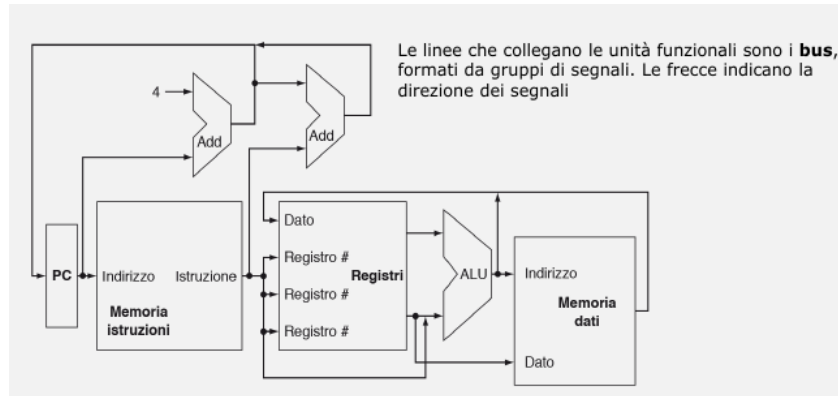


Figure 56: Unità' di elaborazione: panoramica

46 Temporizzazione

Gli elementi di stato hanno in ingresso almeno 2 valori: il valore da scrivere nell'elemento e il clock. Il clock determina quando lo stato puo' essere aggiornato. La metodologia di temporizzazione specifica quando lo stato viene aggiornato. Due metodologie:

- Temporizzazione sensibile ai livelli: lo stato viene aggiornato quando il clock e' a livello alto (o basso). Usata nel latch D.
- Temporizzazione sensibile ai fronti: lo stato viene aggiornato in corrispondenza di un fronte del segnale del clock (fronte di salita o discesa). Usata nei flip-flop

Con una metodologia sensibile ai fronti, e' possibile leggere il contenuto di un registro, inviarlo attraverso una rete combinatoria e scrivere lo stesso registro nello stesso ciclo di clock. Assumiamo che lo stato sia gia' aggiornato sul fronte di salita.

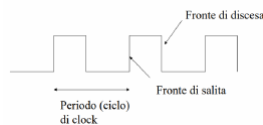


Figure 57: Temporizzazione

47 CPU singolo ciclo

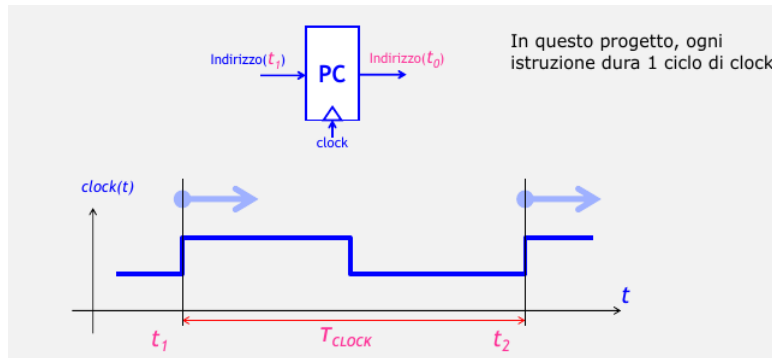


Figure 58: CPU singolo ciclo = 1 istruzione per ciclo di clock

48 Realizzazione di un'unità di elaborazione

Per implementare l'operazione di fetch di un'istruzione, si utilizzano questi elementi:

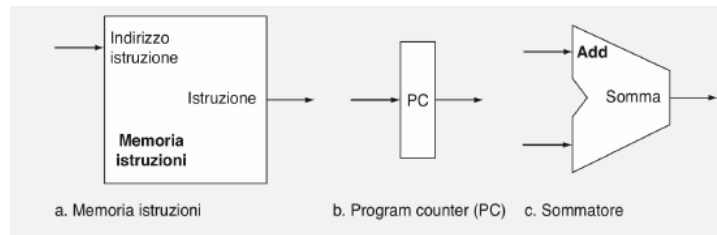


Figure 59: Elementi fetch

Il PC contiene l'indirizzo dell'istruzione corrente. Deve essere aggiornato al valore $PC+4$ (istruzione successiva). Per questo si usa un sommatore: circuito combinatorio che calcola la somma dei due ingressi a 32 bit. La memoria istruzioni e' il circuito combinatorio che realizza la lettura di una istruzione dalla memoria centrale all'indirizzo specificato.

49 Fetch di un'istruzione

Il contenuto del PC rappresenta l'ingresso dell'unità Memoria Istruzioni la quale legge l'istruzione da eseguire. Nello stesso tempo viene calcolato $PC+4$.

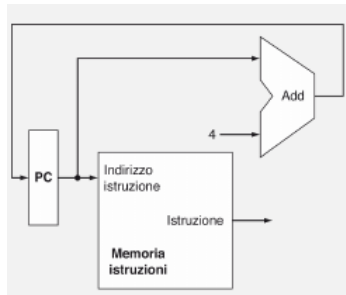


Figure 60: Fetch di un istruzione

49.1 Fase di decodifica

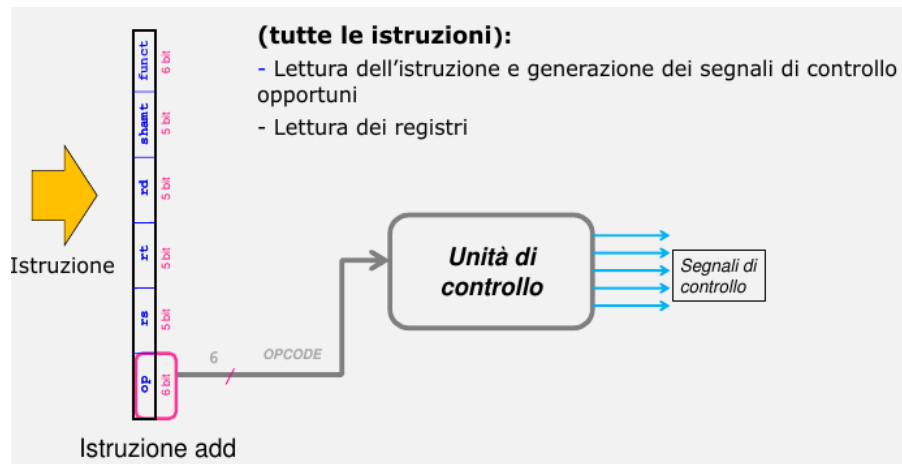


Figure 61: Fase di decodifica

49.2 Esecuzione istruzioni formato R

Sequenza di azioni:

1. Lettura dei registri sorgente
2. Operazione dell'ALU
3. Scrittura del risultato nel registro destinazione

49.3 Banco dei registri (register file)

I 32 registri generali sono raccolti in una unità detta register file, o banco dei registri. Il banco dei registri permette di leggere e scrivere in un registro

specificando il numero del registro. I numero dei 2 registri sorgente sono gli ingressi alle porte di lettura. Il numero del registro destinazione e' l'ingresso alla porta di scrittura. Il segnale di controllo RegWrite abilita la scrittura nel registro del dato in ingresso, La scrittura deve essere quindi indicata esplicitamente, a differenza della lettura.

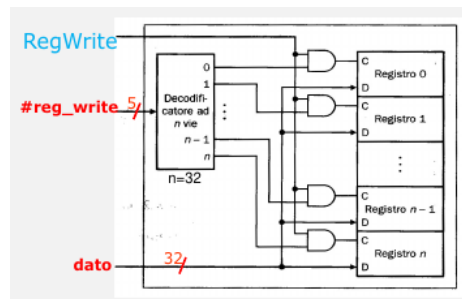


Figure 62: Banco dei registri

49.3.1 Lettura dei registri

L'operazione comporta la lettura di 2 registri. Si usano 2 multiplexer (MUX) per la selezione del registro.

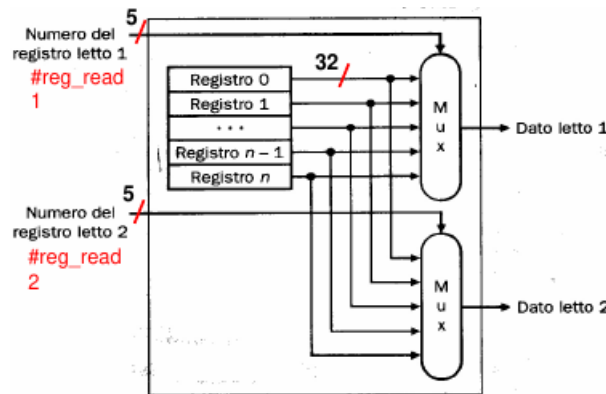


Figure 63:

49.3.2 Scrittura di un registro

L'operazione di scrittura in un registro con ingresso C, D:

- Ingresso C: decoder(i) AND RegWrite. Il clock e' implicito nel disegno.
- Ingresso D: dato da scrivere nel registro (32 bit)

49.4 ALU: unita' aritmetico logica

Ha due ingressi di 32 bit, produce un risultato su 32 bit, in aggiunta restituisce un bit (zero) che vale 1 se il risultato dell'operazione e' 0. I segnali di controllo ALUs servono a specificare l'operazione da effettuare (es: add, and..)

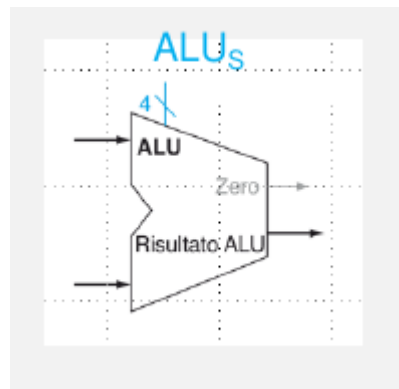


Figure 64: ALU

49.4.1 Esecuzione di una istruzione di tipo R: lettura registri

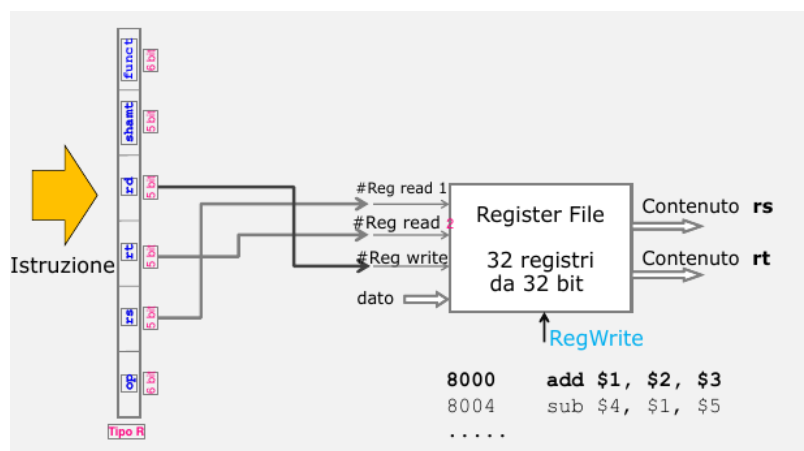


Figure 65:

49.4.2 Esecuzione di una istruzione di tipo R: ALU

I dati vengono letti da Register File e immessi nella ALU. La ALU viene istruita tramite i segnali ALUs.

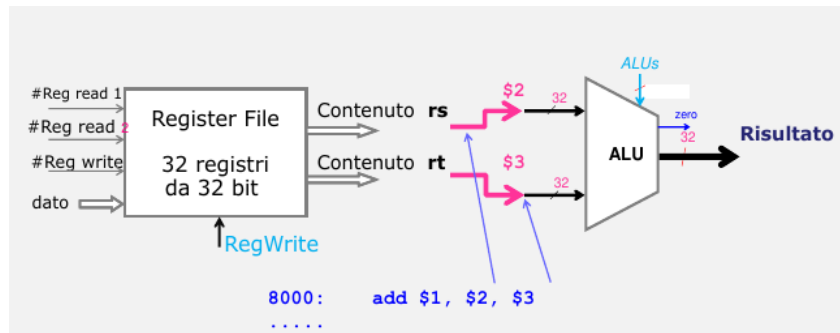


Figure 66:

49.4.3 Esecuzione di una istruzione di tipo R: scrittura risultato

Il risultato dell'operazione viene scritto nel registro destinazione. Il segnale di controllo RegWrite abilita la scrittura nel registro.

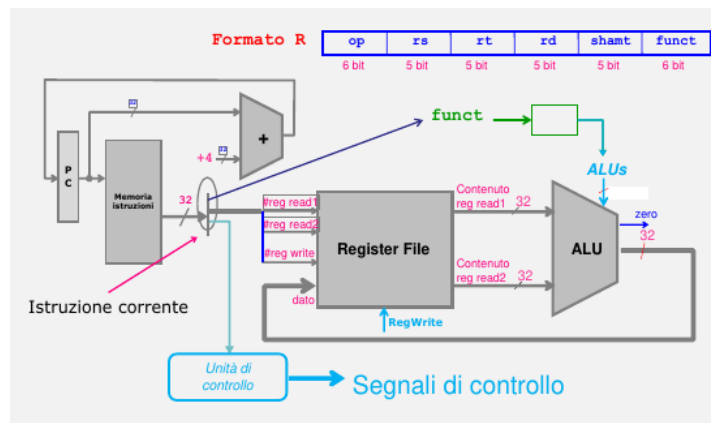


Figure 67: Istruzioni di tipo R schema complessivo

50 Istruzioni di trasferimento dati: lw (sw)

50.1 Unità funzionali coinvolte (lw/sw)

Oltre al Register file e ALU si utilizza: memoria dati e unità di estensione del segno.

La memoria dati e' un circuito combinatorio usato per accedere in memoria centrale. Ha in ingresso 2 segnali di controllo: MemRead e MemWrite. Indicano quale operazione deve essere effettuata. L'unità estensione del segno inserisce nei 16 bit più significativi il bit di segno della costante (o tutti 1 o tutti 0)

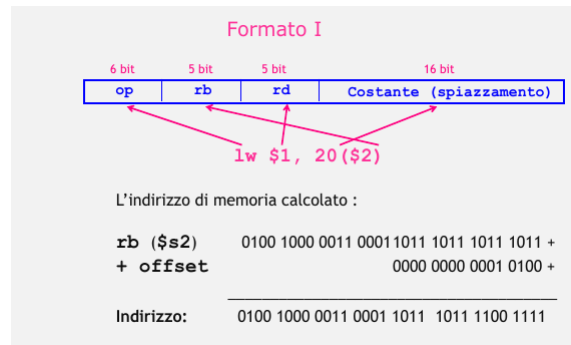


Figure 68: Istruzioni di trasferimento

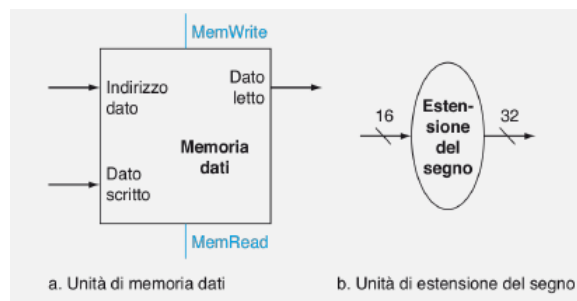


Figure 69:

51 Istruzione lw

1. Lettura del registro base dal Register File
2. Calcola l'indirizzo di memoria: registro base + spiazzamento
3. Lettura del dato dalla memoria all'indirizzo calcolato
4. Scrittura del dato nel registro destinazione

51.1 lw: calcolo indirizzo memoria

51.2 lw: lettura dato e scrittura

51.3 lw: schema complessivo

52 Istruzione sw

1. Lettura del registro base e del registro sorgente
2. Calcolo dell'indirizzo di memoria: registro base + spiazzamento

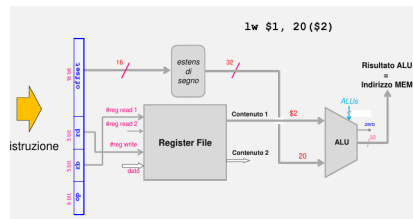


Figure 70:

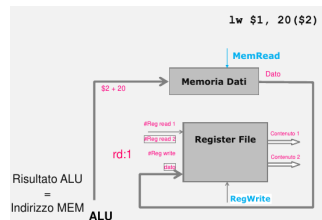


Figure 71:

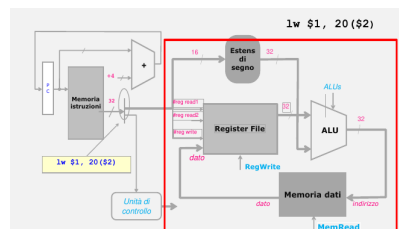


Figure 72:

3. Scrittura del contenuto del registro sorgente in memoria

53 Schema per istruzioni tipo R e lw/sw

Fino ad ora abbiamo visto separatamente i vari circuiti che implementano le diverse istruzioni. Tuttavia, si può verificare che ad un dato dispositivo i dati in input provengano da fonti diverse, per cui nasce il problema di dover selezionare la fonte opportuna.

- Il dato da scrivere nel registro può derivare o dalla memoria o dalla ALU
- Un operando della ALU può essere un registro o una costante

Quando si deve effettuare una selezione degli ingressi, si usa un multiplexer. Si aggiungono 2 multiplexer e i corrispondenti segnali di controllo.

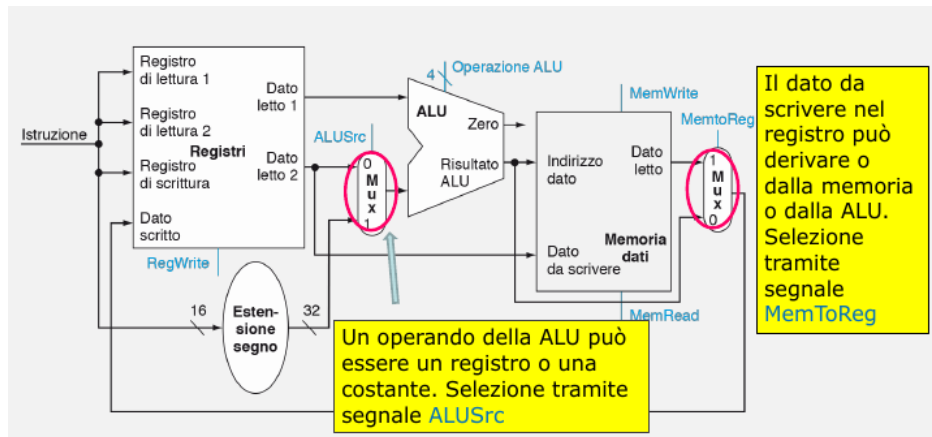


Figure 73:

54 Istruzione beq

1. Calcolo dell'indirizzo di salto:
 - Estensione dell'offset su 32 bit
 - Moltiplicazione per 4 dell'offset
 - Somma del PC con l'estensione del segno
2. Controllo se il contenuto dei registri e' uguale
3. Se uguale, l'indirizzo della prossima istruzione sara' quello calcolato altrimenti sara' l'indirizzo in sequenza.

54.1 Istruzione beq: calcolo indirizzo di salto

Oltre alla ALU e al circuito per l'estensione del segno, si usa un sommatore per determinare l'indirizzo del salto. Inoltre viene effettuata una operazione di shift a sinistra di 2 posizioni per calcolare l'indirizzo al byte. Si deve aggiungere un sommatore perche' la ALU e' gia' impegnata (all'interno del ciclo di clock, ogni unita' e' usata una sola volta)

L'indirizzo da scrivere nel PC (Program Counter) puo' essere quello in sequenza oppure quello calcolato dall'istruzione di branch. Viene aggiunto un multiplexer e il segnale PCSrc. Il bit Zero viene usato dalla logica di controllo dei dati salti per determinare il valore di PCSrc. Oltre alla ALU e al circuito per l'estensione del segno, si usa un sommatore per determinare l'indirizzo del salto. Inoltre viene effettuata una operazione di shift a sinistra di 2 posizioni per calcolare l'indirizzo al byte.

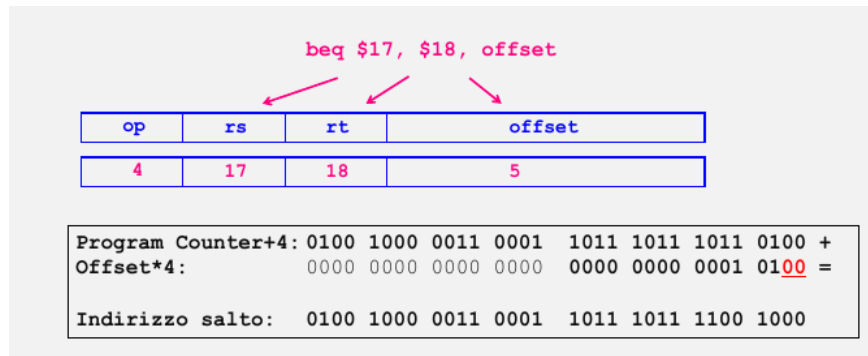


Figure 74:

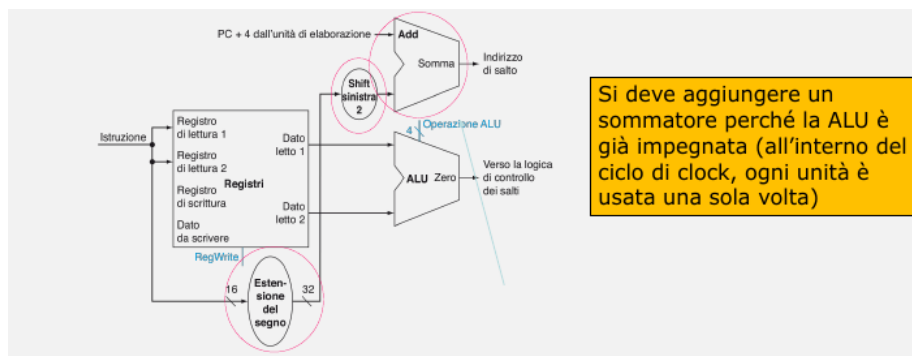


Figure 75:

54.2 beq: confronto dei registri

La ALU effettua la sottrazione fra i 2 registri. Se i contenuti sono uguali, in uscita si ha bit Zero=1. Il bit Zero viene utilizzato dalla logica di controllo dei salti per stabilire se nel PC debba essere scritto il valore dell'indirizzo in sequenza o quello calcolato.

54.3 Indirizzo da scrivere nel PC

55 Istruzione jump

I 26 bit stanno a significare l'indirizzo assoluto in termini di parole. Costruito in due passi:

- Costruzione parte bassa indirizzo (28 bit): $addr = indirizzo * 4$. Si effettua uno shift a sinistra di 2 posizioni, per cui i 2 bit meno significativi sono posti a 0.

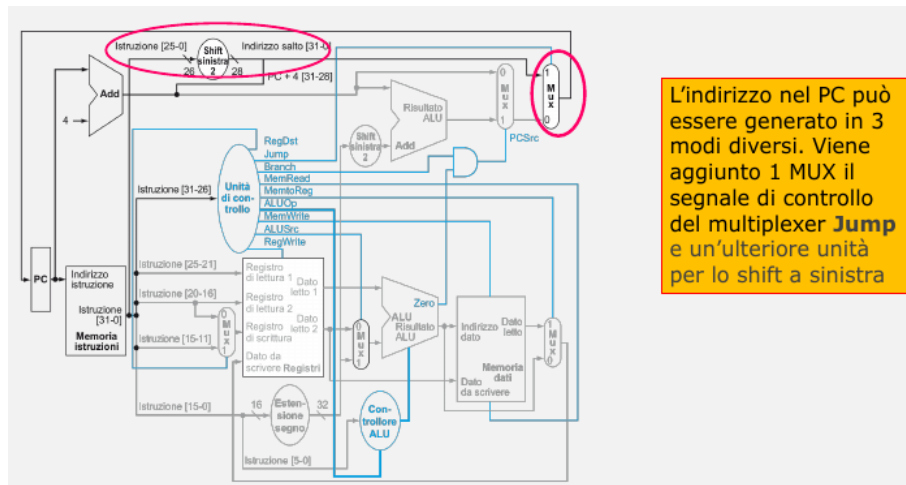


Figure 78:

56 Unità' di Controllo (UC)

Abbiamo costruito un'unità di elaborazione a ciclo singolo. Non abbiamo ancora parlato dell'unità di controllo. La unità di controllo invia i segnali di controllo alla unità di elaborazione. Iniziamo considerando una piccola unità, detta unità di controllo della ALU

56.1 Unita' di controllo della ALU

Abbiamo visto che la Alu e' utilizzata non solo per operazioni aritmetico-logiche ma anche per le istruzioni di trasferimento dati e salto condizionato. L'operazione viene specificata dalla configurazione dei 4 bit di controllo ALUs:

- 0000 and
- 0001 or
- 0010 add/somma (diverse istruzioni, viene usata anche per calcolare l'indirizzo di una operazione)
- 0110 sottrazione (diverse istruzioni, nell'istruzione di salto condizionato)
- 0111 set less than
- 1100 nor (non rientra tra le operazioni che consideriamo)

Per le istruzioni lw e sw, la ALU esegue una somma per calcolare l'indirizzo di memoria; per le istruzioni di tipo R (add, sub, and, or, slt) effettua le prime 5 operazioni (non è considerata); per l'istruzione di salto condizionato (beq) deve essere effettuato una sottrazione. Nota: sarebbero sufficienti 3 bit per

ALUS (il bit piu' significativo e' 0). Manteniamo 4 bit per uniformita' con quanto riportato nel libro H&P.

L'unita' di controllo della ALU e' una rete combinatoria che riceve in ingresso il campo funzione (funct) dell'istruzione e un campo di controllo costituito da 2 bit inviati dall'unita' di controllo principale (ALUop) $ALUs = f(ALUop, funct)$

- $ALUop = 10$: l'operazione dipende dal campo funzione
- $ALUop = 00$: l'operazione e' una somma in una istruzione di lw/sw
- $ALUop = 01$: l'operazione e' una beq

56.2 Unita' di controllo principale (UC)

Dopo aver visto la unita' di controllo della ALU, passiamo a considerare la unita' di controllo principale (UC). Oltre al campo ALUop, la UC genera i segnali di controllo per le unita' funzionali e i multiplexer. I segnali sono generati in funzione del codice operativo. Oltre ad ALUop, vi sono 7 segnali di controllo. Questi segnali comprendono: i segnali di selezione per il controllo dei multiplexer e segnali di comando.